

Министерство цифрового развития, связи и массовых коммуникаций  
Российской Федерации

СибГУТИ

Кафедра вычислительных систем

## Расчетно-графическая работа

«Разработка сетевого приложения «Сетевая игра».  
Мультипоточная реализация сервера с установлением  
соединения с использованием библиотеки PTHREAD»

Выполнил: студент группы ИП-312

Дорогин Н. С.

Проверил: старший преподаватель  
кафедры вычислительных систем

Ревун А. Л.

Новосибирск, 2026

## Содержание

|                                              |           |
|----------------------------------------------|-----------|
| <b>1. Постановка задачи .....</b>            | <b>3</b>  |
| 1.1. Общая формулировка .....                | 3         |
| 1.2. Описание игры .....                     | 3         |
| <b>2. Описание протокола .....</b>           | <b>4</b>  |
| <b>3. Описание реализации .....</b>          | <b>6</b>  |
| 3.1. Общая архитектура .....                 | 6         |
| 3.2. Сетевое взаимодействие .....            | 6         |
| 3.3. Мультипоточная архитектура сервера..... | 7         |
| 3.4. Игровая логика.....                     | 8         |
| 3.5. База данных .....                       | 10        |
| 3.6. Система чата.....                       | 10        |
| 3.7. Командная строка клиента .....          | 11        |
| 3.8. Обработка ошибок и таймауты .....       | 11        |
| 3.9. Сборка и запуск.....                    | 12        |
| <b>4. Сканы работы программы.....</b>        | <b>13</b> |
| <b>5. Текст программы.....</b>               | <b>24</b> |
| 5.1. Клиент-сервер .....                     | 24        |
| 5.2. Библиотека “Lobby” .....                | 46        |
| 5.3. Библиотека “Game” .....                 | 57        |
| 5.4. Библиотека “Chat” .....                 | 76        |
| 5.5. Библиотека “DbContext” .....            | 77        |
| <b>Список источников .....</b>               | <b>87</b> |

# 1. Постановка задачи

## 1.1. Общая формулировка

Программно реализовать настольную игру “Стартап” в виде мультипоточного сервера с установлением соединения с использованием библиотеки PTHREAD.

Проект должен представлять из себя полноценный игровой сервер с регистрацией, авторизацией, лобби и рейтингом игроков. Дополнительно должен присутствовать механизм общения пары игроков в отдельном соединении в обход сервера.

## 1.2. Описание игры

Настольная игра для креативных собеседований в фантастических проектах!

### КОМПЛЕКТАЦИЯ

Карты "Профессий": 50 шт.

Карты "Навыков": 70 шт.

Карты "Эмоций": 50 шт.

Игроки: 3-6 человек

### ПРАВИЛА ИГРЫ

Игра состоит из нескольких этапов (по количеству игроков).

#### Цикл раунда:

##### 1. НАЗНАЧЕНИЕ ВЕДУЩЕГО

Автоматически выбирается работодатель

##### 2. ПРОФЕССИИ ВЕДУЩЕМУ

Работодатель получает 3 карты из колоды "Профессии"

##### 3. ПРЕЗЕНТАЦИЯ ПРОЕКТА

Работодатель описывает фантастический проект и вакансии

#### 4. РАЗДАЧА СОИСКАТЕЛЯМ

1 карта "Эмоции"

4 карты "Навыки"

#### 5. СОБЕСЕДОВАНИЕ (по очереди каждого соискателя)

5.1. Выбор должности

5.2. Презентация себя работодателю (макс. 1 минута)

5.3. Вопросы от остальных игроков

5.4. Вскрытие карт соискателя

5.5. Оценка от 1 до 5 (баллы суммируются)

#### 6. РАСПРЕДЕЛЕНИЕ ВАКАНСИЙ

Полученная профессия дает +3 бонусных очка

Профессия сохраняется у игрока

#### **Завершение раунда**

Использованные навыки и эмоции отправляются на дно колоды

### **КОНЕЦ ИГРЫ**

Игра завершается, когда каждый игрок побывал в роли работодателя.

### **ПОБЕДИТЕЛЬ**

Побеждает игрок, набравший максимальное количество баллов за все раунды.

## **2. Описание протокола**

**TCP (Transmission Control Protocol)** – протокол транспортного уровня модели OSI, основанный на передаче информации с установлением соединения между узлами.

## Основные характеристики

- **Надёжность** – гарантирует доставку всех данных в правильном порядке. Потерянные пакеты пересылаются автоматически.
- **Установление соединения** – перед началом передачи данных стороны выполняют трёхэтапное рукопожатие (three-way handshake): запрос на соединение (SYN), принятие запроса (SYN), подтверждение от запрашивающего, что он оповещён о принятии (ACK).
- **Контроль потока** – использует механизм скользящего окна (sliding window), чтобы отправитель не перегружал получателя.
- **Контроль перегрузки** – адаптивно уменьшает скорость передачи при перегрузках в сети.
- **Дуплексный режим** – данные могут передаваться в обоих направлениях одновременно.
- **Сегментация** – разбивает данные на сегменты оптимального размера для передачи по сети.

| Формат TCP-сегмента |                          |                 |        |                    |
|---------------------|--------------------------|-----------------|--------|--------------------|
| Бит                 | 0 - 3                    | 4 - 7           | 8 - 15 | 16 - 31            |
| 0                   | Порт источника           |                 |        | Порт назначения    |
| 32                  | Номер последовательности |                 |        |                    |
| 64                  | Номер подтверждения      |                 |        |                    |
| 96                  | Смещение данных          | Зарезервировано | Флаги  | Окно               |
| 128                 | Контрольная сумма        |                 |        | Указатель важности |
| 160                 | Опции (необязательное)   |                 |        |                    |
| 160/192+            | Данные                   |                 |        |                    |

Рисунок 1. Формат TCP-сегмента (пакета)

## Преимущества использования TCP:

1. Наилучшая попытка по доставке информации. Надёжность.
2. Сохранения порядка данных.
3. Управление потоком (предотвращает переполнение буферов при интенсивном обмене)

### **Недостатки использования TCP:**

1. Более высокие накладные расходы по сравнению с UDP (заголовок больше, требуется установление соединения).
2. Возможная задержка из-за механизмов контроля перегрузки и повторной отправки.
3. «Головная блокировка» (Head-of-line blocking) – потеря одного пакета задерживает все последующие.

## **3. Описание реализации**

### **3.1. Общая архитектура**

Разработанное сетевое приложение представляет собой клиент-серверную систему с мультипоточной реализацией сервера на основе библиотеки Pthread. Система включает следующие основные компоненты:

- Сервер (server.cpp) – центральный узел, обрабатывающий подключения клиентов, управление авторизацией, лобби и игровыми сессиями.
- Клиент (client.cpp) – пользовательское приложение, обеспечивающее взаимодействие с сервером, участие в игре и чате.
- Библиотека игры (game.h/cpp) – реализует игровую логику, карты, профессии, навыки, эмоции и систему оценки.
- Библиотека лобби (lobby.h/cpp) – управляет игровыми комнатами и потоками игроков внутри игры.
- Библиотека контекста БД (dbcontext.h/cpp) – обеспечивает взаимодействие с PostgreSQL для хранения пользователей, рейтинга и информации о лобби.
- Библиотека чата (chat.h/cpp) – реализует асинхронный обмен сообщениями между игроками в обход сервера.

### **3.2. Сетевое взаимодействие**

#### Транспортный протокол:

Для всех сетевых взаимодействий используется протокол TCP, что обеспечивает:

Гарантированную доставку всех сообщений

Сохранение порядка пакетов

Надёжную передачу игровых данных без потерь

#### Формат сообщений:

Для унификации обмена данными между клиентом и сервером разработан специальный формат сообщений:

`"output|request|player_status"`

Где:

*output* – основное содержимое сообщения (текст, данные, результаты операций)

*request* – тип запроса (login, register, getplayers, makelob, join и т.д.)

*player\_status* – текущий статус игрока (WAIT\_ACCEPT, PRE\_TO\_PLAY, EMPLOYER, ANSWERING и т.д.)

Разделителем служит символ |, что позволяет легко парсить сообщения на обеих сторонах. Функции `cli_decode_msg()` и `ser_decode_msg()` обеспечивают унифицированное кодирование и декодирование сообщений.

#### Соединения:

Приложение использует три типа TCP-соединений:

- Основное соединение (клиент ↔ сервер) – для авторизации, командной строки и координации игр.
- Игровое соединение (клиент ↔ лобби) – для обмена игровыми сообщениями внутри одной игровой сессии.
- Чат-соединение (клиент ↔ клиент) – прямое P2P-соединение между игроками для обмена сообщениями в обход сервера.

### 3.3. Мультипоточная архитектура сервера

Сервер реализован с использованием библиотеки *Pthread* и включает следующие типы потоков:

#### Главный поток (main):

Создаёт серверный сокет и ожидает входящие соединения.

Для каждого нового клиента создаёт отдельный поток `user_thread`.

Инициализирует подключение к базе данных PostgreSQL.

#### Поток пользователя (user\_thread):

Обслуживает одного подключённого клиента.

Обрабатывает запросы авторизации и регистрации.

Выполняет команды командной строки (ps, psa, rt, ls, mkl, join, chats, chat, mes, clchat, about, exit).

При создании лобби запускает отдельный поток lobby\_thread.

#### Поток лобби (lobby\_thread):

Создаётся для каждой игровой комнаты (лобби).

Управляет игровым процессом в рамках одного лобби.

При подключении игроков создаёт для каждого отдельный поток player\_thread.

Синхронизирует доступ к общим данным через мьютексы.

#### Поток игрока (player\_thread):

Обслуживает одного игрока внутри лобби.

Обрабатывает игровые события: выбор вакансии, написание истории, ответы на вопросы, выставление оценок.

Взаимодействует с объектом Game для выполнения игровой логики.

#### Синхронизация потоков:

Для обеспечения потокобезопасности используются мьютексы (pthread\_mutex\_t):

- Мьютекс лобби – защищает доступ к списку игроков в лобби, очередям вопросов и общим игровым данным.
- Мьютекс базы данных – предотвращает одновременные запросы к PostgreSQL из разных потоков.

Критические секции оформлены с минимальной длительностью для предотвращения взаимных блокировок (deadlock).

### 3.4. Игровая логика

#### Класс Game:

- Основной класс, реализующий игровую механику:



- Колоды карт – профессии (50 шт.), навыки (70 шт.), эмоции (50 шт.). Перемешиваются при инициализации.
- Управление игроками – добавление, удаление, изменение статусов, подсчёт очков.
- Игровой цикл – последовательная смена статусов: PRE → START → JOB\_MAKE → P\_PRE → P\_MAKE → QUESTIONS → P\_OPEN → SCORES → JOB\_CHOICE → START (следующий раунд) → OVER.
- Раздача карт – работодатель получает 3 профессии, соискатели – 4 навыка и 1 эмоцию.
- Система вопросов – поддержка очереди вопросов от игроков, последовательная обработка ответов.
- Оценка и рейтинг – суммирование оценок, начисление бонусов за профессии, изменение рейтинга по окончании игры.

#### Статусная модель:

Игра использует две системы статусов (таблица 1, таблица 2):

*Таблица 1. Статусы игры (game\_status)*

| <b>game_status</b> | <b>Значение</b>                   |
|--------------------|-----------------------------------|
| PRE                | набор игроков в лобби             |
| FULL               | лобби заполнено                   |
| START              | игра начинается                   |
| JOB_MAKE           | работодатель придумывает историю  |
| P_PRE              | соискатель готовится              |
| P_MAKE             | соискатель выступает              |
| QUESTIONS          | режим вопросов                    |
| P_OPEN             | вскрытие карт                     |
| SCORES             | выставление оценок                |
| JOB_CHOICE         | работодатель выбирает сотрудников |
| OVER               | игра окончена                     |

*Таблица 2. Статусы игрока (player\_status)*

| <b>player_status</b> | <b>Значение</b>           |
|----------------------|---------------------------|
| WAIT_ACCEPT          | ожидание принятия в лобби |
| PRE_TO_PLAY          | готовится к игре          |
| READY_TO_PLAY        | готов к началу            |
| WAITING              | ожидание своей очереди    |

|             |                      |
|-------------|----------------------|
| EMPLOYER    | текущий работодатель |
| ANSWERING   | отвечает на вопросы  |
| QUESTIONING | задаёт вопросы       |
| SCORING     | выставляет оценку    |
| LEFT        | покинул игру         |

### 3.5. База данных

Для хранения информации используется PostgreSQL. Структура БД включает:

Таблица users:

- id – первичный ключ, автоинкремент
- login – уникальное имя пользователя
- password – хешированный пароль (libsodium)
- online – статус онлайн/оффлайн
- score – рейтинг игрока
- address – IP-адрес клиента
- port – порт для чата

Таблица games:

- id – первичный ключ
- name – название лобби
- size – максимальное количество игроков
- busy – текущее количество игроков
- port – порт лобби
- began – статус начала игры
- creator – ID создателя (ссылка на users.id)

Для защиты паролей используется библиотека libsodium с алгоритмом Argon2 – криптостойкое хеширование с автоматической генерацией соли.

### 3.6. Система чата

Чат между игроками реализован в обход сервера по следующей схеме:

1. При входе в систему каждый клиент запускает msg\_accept\_thread, который слушает входящие чат-соединения на свободном порту.

2. Клиент сообщает серверу свой IP-адрес и порт для чата через команду setchatport.
3. При отправке сообщения клиент запрашивает у сервера IP и порт целевого игрока.
4. Клиент устанавливает прямое TCP-соединение с целевым игроком и отправляет сообщение.
5. Принятое сообщение записывается в файл чата и выводится на экран получателя.
6. Такой подход разгружает сервер и обеспечивает низкую задержку при обмене сообщениями.

### 3.7. Командная строка клиента

Клиент предоставляет интерактивную командную строку (таблица 3)

*Таблица 3. Команды клиента*

| Команда | Описание                       |
|---------|--------------------------------|
| ps      | Показать список игроков онлайн |
| psa     | Показать список всех игроков   |
| rt      | Показать рейтинг               |
| ls      | Показать список лобби          |
| mkl     | Создать лобби                  |
| join    | Присоединиться к лобби         |
| chats   | Показать список чатов          |
| chat    | Показать чат с игроком         |
| mes     | Отправить сообщение игроку     |
| clchat  | Очистить чат с игроком         |
| about   | Показать информацию об игре    |
| help    | Показать справку               |
| exit    | Выйти из игры                  |

### 3.8. Обработка ошибок и таймауты

Для операций ввода-вывода установлены таймауты через setsockopt(SO\_RCVTIMEO).

При разрыве соединения игрок корректно удаляется из лобби и базы данных.

Предусмотрена обработка сигнала SIGPIPE для предотвращения аварийного завершения при записи в закрытый сокет.

Все операции с базой данных защищены мьютексами и обернуты в try-catch блоки.

### 3.9. Сборка и запуск

Проект собирается с помощью Makefile. Основные цели:

#### Сборка сервера

```
bash
make server
```

#### Сборка клиента

```
bash
make client
```

#### Запуск сервера

```
bash
make run-ser
```

#### Запуск клиента

```
bash
make run-cli
```

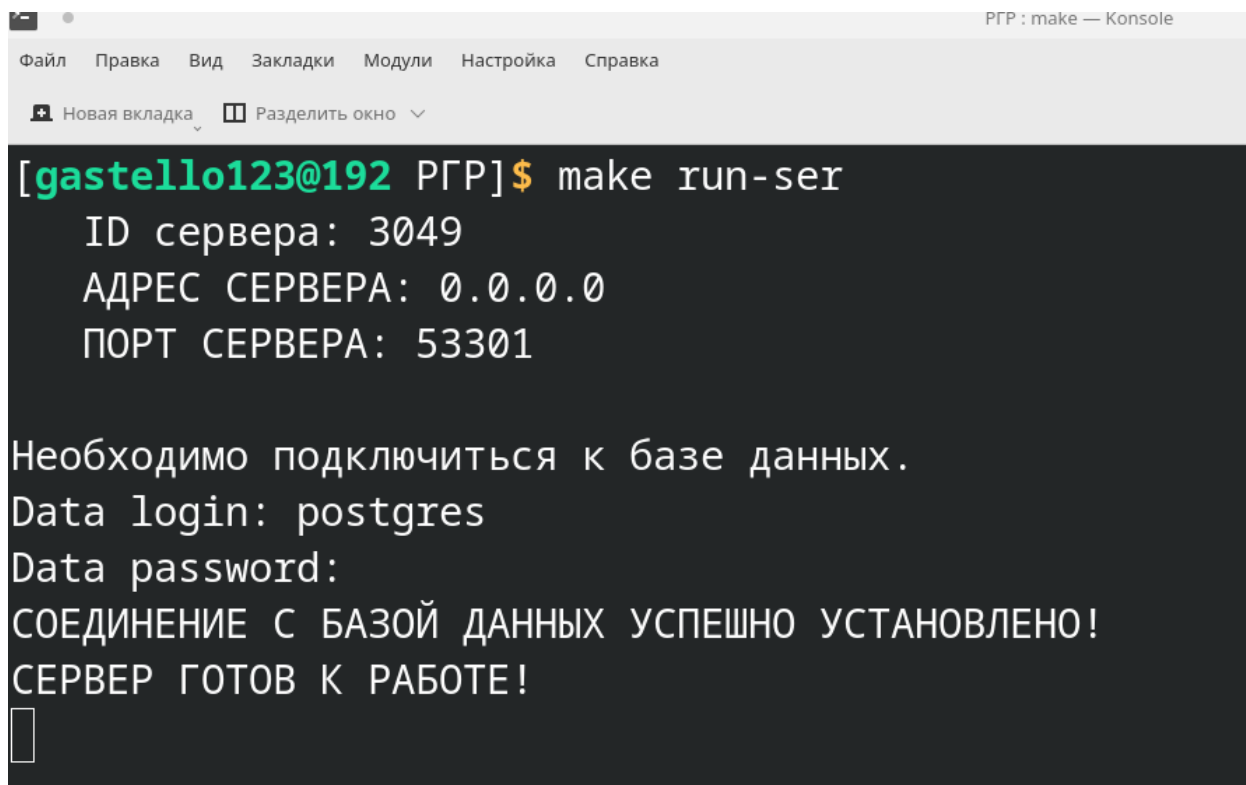
#### Очистка объектных файлов

```
bash
make clean
```

#### Зависимости:

- компилятор с поддержкой C++17
- библиотеки libpqxx (PostgreSQL)
- libsodium (криптография)
- pthread (многопоточность).

## 4. Сканы работы программы



```
RGP : make — Konsole
Файл  Правка  Вид  Закладки  Модули  Настройка  Справка
+ Новая вкладка  Разделить окно
[gastello123@192 RGP]$ make run-ser
ID сервера: 3049
АДРЕС СЕРВЕРА: 0.0.0.0
ПОРТ СЕРВЕРА: 53301

Необходимо подключиться к базе данных.
Data login: postgres
Data password:
СОЕДИНЕНИЕ С БАЗОЙ ДАННЫХ УСПЕШНО УСТАНОВЛЕНО!
СЕРВЕР ГОТОВ К РАБОТЕ!
```

*Рисунок 2. Запуск сервера*

```
[gastello123@192 PGP]$ make run-cli
=====
Добро пожаловать в СТАРТАП!
=====

ВВЕДИТЕ АДРЕС СЕРВЕРА (или exit, чтобы выйти): localhost
ВВЕДИТЕ ПОРТ СЕРВЕРА: 53301

Логин(L)/Регистрация(R)/Выйти(E): L

Логин: Tim
Пароль:
Неверный логин или пароль!

Логин(L)/Регистрация(R)/Выйти(E): R
Придумайте логин: Tim
Придумайте пароль:
Повторите пароль:
Регистрация пройдена!

Логин(L)/Регистрация(R)/Выйти(E): L

Логин: Tim
Пароль:

Вы успешно вошли на сервер!
ПОДСКАЗКА: для просмотра доступных команд введите help

[Tim]$
```

*Рисунок 3. Регистрация/Логин*

```
[Tim]$ help  
  
'ps' - показать список игроков онлайн.  
'psa' - показать список всех игроков.  
'rt' - показать рейтинг.  
'ls' - показать список лобби.  
'mkl' - создать лобби.  
'join' - присоединиться к лобби.  
'chats' - показать чаты.  
'chat' - показать чат с игроком.  
'mes' - написать сообщение игроку.  
'clchat' - очистить чат с игроком.  
'about' - об игре.  
'exit' - выйти.  
  
[Tim]$
```

*Рисунок 4. Вывод списка команд*

```

[Tim]$ about
О ИГРЕ "СТАРТАП"
=====

Настольная игра для креативных собеседований в фантастических проектах!

--- КОМПЛЕКТАЦИЯ ---

Карты "Профессий": 50 шт.
Карты "Навыков": 70 шт.
Карты "Эмоций": 50 шт.
Игроки: 3-6 человек

--- ПРАВИЛА ИГРЫ ---

Игра состоит из нескольких этапов/ранудов (по количеству игроков).

ЦИКЛ РАУНДА:
\////////////////////////////////////
  1. НАЗНАЧЕНИЕ ВЕДУЩЕГО
    Автоматически выбирается работодатель

  2. ПРОФЕССИИ ВЕДУЩЕМУ
    Работодатель получает 3 карты из колоды "Профессии"

  3. ПРЕЗЕНТАЦИЯ ПРОЕКТА
    Работодатель описывает фантастический проект и вакансии

  4. РАЗДАЧА СОИСКАТЕЛЯМ

    1 карта "Эмоции"

    4 карты "Навыки"

  5. СОБЕСЕДОВАНИЕ (по очереди каждого соискателя)
    5.1. Выбор должности
    5.2. Презентация себя работодателю (макс. 1 минута)
    5.3. Вопросы от остальных игроков
    5.4. Вскрытие карт соискателя
    5.5. Оценка от 1 до 5 (баллы суммируются)

  6. РАСПРЕДЕЛЕНИЕ ВАКАНСИЙ

    Полученная профессия дает +3 бонусных очка

    Профессия сохраняется у игрока
    \////////////////////////////////////
ЗАВЕРШЕНИЕ РАУНДА
Использованные навыки и эмоции отправляются на дно колоды

КОНЕЦ ИГРЫ

Игра завершается, когда каждый игрок побывал в роли работодателя.

```

*Рисунок 5. Вывод информации об игре*



```
[Tim]$ ps
=====
          ID          Логин
=====
          8          Tim
-----

[Tim]$ psa
=====
          ID      Логин      Статус
=====
          1      Nekit      Не в сети
-----
          2      Maks      Не в сети
-----
          3      Roma      Не в сети
-----
          4      Anton      Не в сети
-----
          5      Ivan      Не в сети
-----
          6      Aleksey      Не в сети
-----
          7      RevAL      Не в сети
-----
          8      Tim      Онлайн
-----
```

*Рисунок 6. Вывод онлайн и всех игроков*

```
[Tim]$ rt
```

| Место | ID | Логин   | Рейтинг |
|-------|----|---------|---------|
| 1)    | 1  | Nekit   | 299     |
| 2)    | 2  | Maks    | 61      |
| 3)    | 3  | Roma    | 44      |
| 4)    | 4  | Anton   | 7       |
| 5)    | 6  | Aleksey | 3       |
| 5)    | 7  | RevAL   | 3       |
| 6)    | 5  | Ivan    | 0       |
| 6)    | 8  | Tim     | 0       |

Рисунок 7. Вывод рейтинга

```
[Tim]$ ls
```

| ID | Название | Создатель | К-во игроков | Игра началась |
|----|----------|-----------|--------------|---------------|
|----|----------|-----------|--------------|---------------|

```
[Tim]$ mkl
Введите название лобби (или exit для отмены): NEWLOB
Введите количество игроков (3-6, или 0 для отмены): 3
Ваше лобби 'NEWLOB' успешно создано!
```

```
[Tim]$ ls
```

| ID  | Название | Создатель | К-во игроков | Игра началась |
|-----|----------|-----------|--------------|---------------|
| 144 | NEWLOB   | Tim       | 0/3          |               |

Рисунок 8. Создание лобби

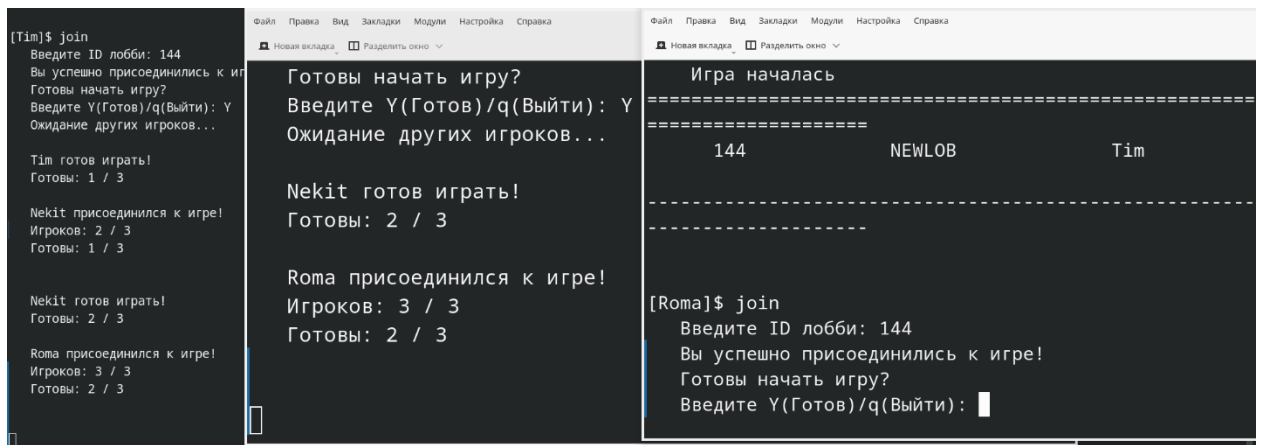


Рисунок 9. Присоединение к лобби

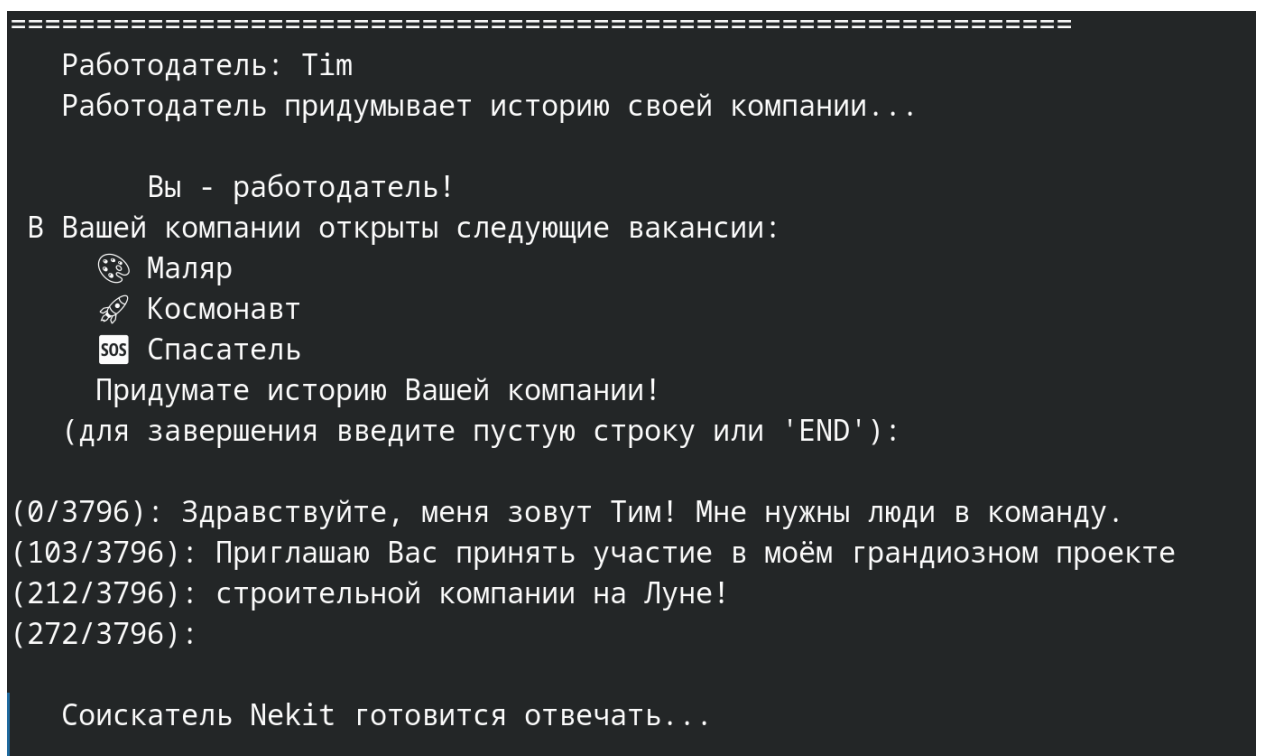


Рисунок 10. Работодатель придумывает историю

```

Работодатель: Tim
Работодатель придумывает историю своей компании...

История:
    Здравствуйте, меня зовут Тим! Мне нужны люди в команду.
    Приглашаю Вас принять участие в моём грандиозном проекте
    строительной компании на Луне!
-----
Вакансии:
-----
1) 🎨 Маляр
2) 🚀 Космонавт
3) 🆘 Спасатель
-----

    Ваша очередь на собеседование!
Эмоция: Вы хотите много денег.
Навыки:
- У меня хорошее чувство ритма.
- Я умею выявлять преимущества положения.
- Я могу найти всё, что угодно в интернете.
- Я умею метко стрелять.

Введите номер вакансии, на которую вы претендуете (1 - 3): 1

Работодатель готов Вас выслушать. Введите 'Y', когда будете готовы отвечать на собеседовании: y
Минута пошла! Удачи!
(для завершения введите пустую строку или 'END'):

(0/3796): Здравствуйте! Меня зовут Никита и я претендую на вакансию маляра!
(123/3796): У Вас же много платят малярам?
(181/3796): У меня отличное чувство ритма и меткость, которые не подведут меня в работе.
(323/3796): Я покрашу Вам стены так, что люди глаз оторвать не смогут.
(431/3796): Я сумею грамотно воспользоваться преимуществами ландшафта Луны
(553/3796): и выберу максимально подходящую палитру цветов для ваших построек!
(680/3796): А если мне будет не хватать каких-либо компетенций, я легко подчерпну что мне надо из интернета!
(858/3796): А аванс у Вас какой?
(897/3796):

Время для вопросов.

```

*Рисунок 11. Соискатель готовится к собеседованию*

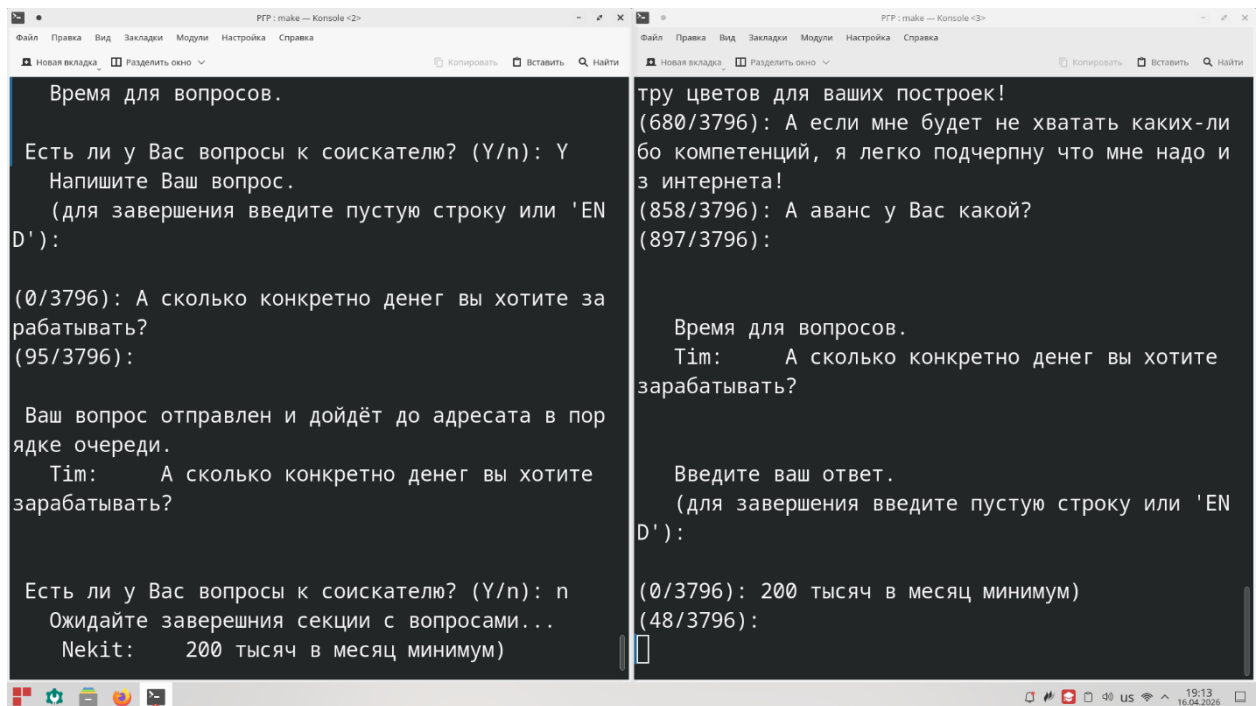


Рисунок 12. Соискатель отвечает на вопросы

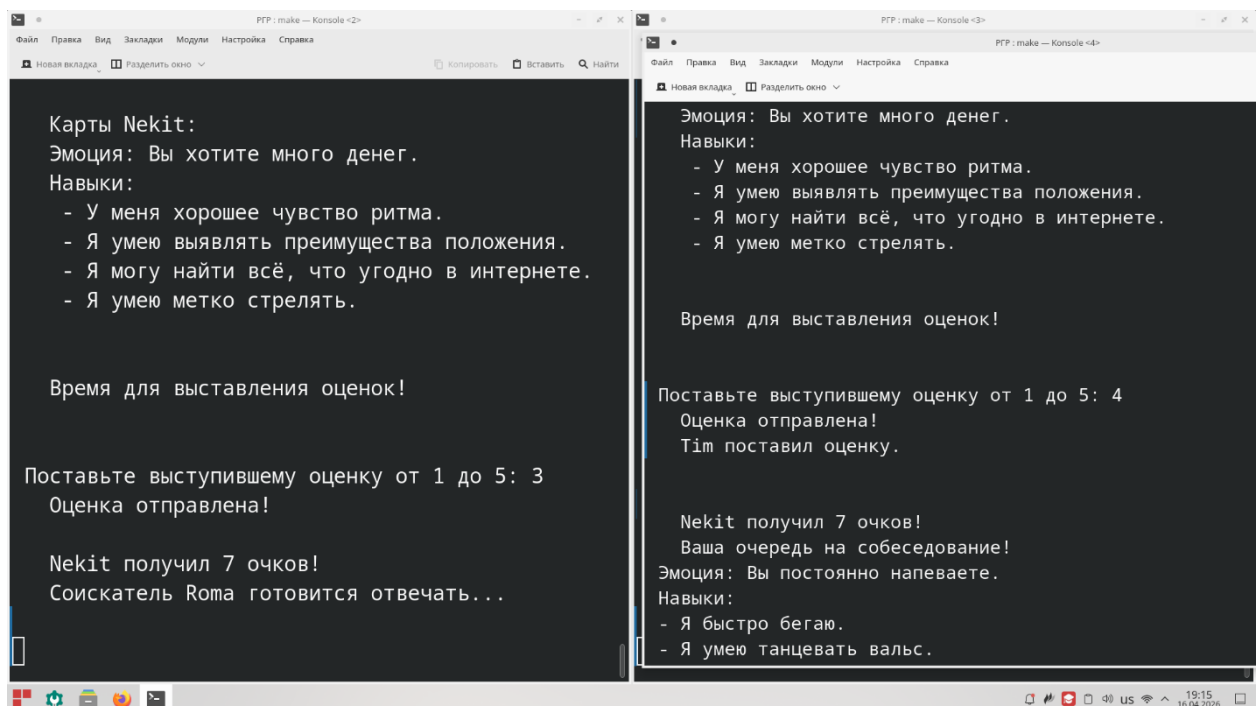


Рисунок 13. Выставление оценок соискателю

```
Введите Ваш выбор в формате: номер_вакансии:номер_игрока (через запятую)
Пример: 1:2,2:1,3:3
  Вакансия 1: 📻 Маляр
Претенденты: 9) Nekit, 11) Roma

  Вакансия 2: 🚀 Космонавт
Претенденты: нет

  Вакансия 3: 🆘 Спасатель
Претенденты: нет

Введите ваш выбор: 1:9

Вакансия "📻 Маляр" достаётся Nekit (выбор работодателя)!
Вакансия "🚀 Космонавт" никому не досталась (нет претендентов)
Вакансия "🆘 Спасатель" никому не досталась (нет претендентов)

=====ИГРОКИ=====
Tim (): 0
Nekit (📻 Маляр): 10
Roma (): 10
=====

Раунд 2:

Работодатель: Nekit
Работодатель придумывает историю своей компании...
```

Рисунок 14. Работодатель выбирает соискателей

```
Nekit получил 10 очков!
Работодатель Roma выбирает сотрудников...
Вакансия 1: 📻 Радиоведущий
Претенденты: 8) Tim, 9) Nekit

Вакансия 2: 🏠 Домработница
Претенденты: нет

Вакансия 3: 🚔 Полицейский
Претенденты: нет

Вакансия "📻 Радиоведущий" достаётся Tim (выбор работодателя)!
Вакансия "🏠 Домработница" никому не досталась (нет претендентов)
Вакансия "🚔 Полицейский" никому не досталась (нет претендентов)

=====
ИГРА ОКОНЧЕНА!
=====

Tim - ПОБЕДИТЕЛЬ! Очки: 23, изменение рейтинга: +15
Nekit - очки: 20, изменение рейтинга: -2
Roma - очки: 20, изменение рейтинга: -2
=====

[Tim]$
```

```
РГР: make — Konsole «4»
Файл  Правка  Вид  Закладки  Модули  Настройка  Справка
НОВАЯ ВКЛАДКА  Разделить окно

Введите ваш выбор: 1:8

Вакансия "📻 Радиоведущий" достаётся Tim (выбор работодателя)
Вакансия "🏠 Домработница" никому не досталась (нет претендентов)
Вакансия "🚔 Полицейский" никому не досталась (нет претендентов)

=====
ИГРА ОКОНЧЕНА!
=====

Tim - ПОБЕДИТЕЛЬ! Очки: 23, изменение рейтинга: +15
Nekit - очки: 20, изменение рейтинга: -2
Roma - очки: 20, изменение рейтинга: -2
=====

[Roma]$
```

Рисунок 15. Конец игры

|                                                                                                   |                                                                 |
|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| 'about' - об игре.<br>'exit' - выйти.<br><br>[Tim]\$ chats<br>Нет активных чатов<br><br>[Tim]\$ █ | [Nekit]\$ chats<br>Активные чаты:<br>1) Maks<br><br>[Nekit]\$ █ |
|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|

*Рисунок 16. Запрос списка чатов*

```

[Nekit]$ ps
=====
ID                Логин
=====
1                 Nekit
-----
3                 Рома
-----
8                 Tim
-----

[Nekit]$ mes
Введите ник игрока: Anton
Игрок Anton не в сети.

[Nekit]$ mes
Введите ник игрока: Рома
Введите ваше сообщение:
(для завершения введите пустую строку или 'END'):

(0/3796): Привет Рома!█

```

*Рисунок 17. Отправка сообщения игроку*

```

[Tim]$ exit
=====Выход из игры!=====
    Вы уверены, что хотите выйти? (y/N): y
=====Выход с сервера!=====

    ВВЕДИТЕ АДРЕС СЕРВЕРА (или exit, чтобы выйти): exit
=====Выход из игры!=====
    Вы уверены, что хотите выйти? (y/N): y
=====Игра закрывается!=====
[gastello123@192 PGP]$ █

```

Рисунок 18. Завершение работы клиента

## 5. Текст программы

### 5.1. Клиент-сервер

client.cpp

```

// КЛИЕНТ (ИСХОДНЫЙ КОД)
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <arpa/inet.h>
#include <time.h>
#include <string.h>
#include <unistd.h>
#include "game.h"
#include <termios.h>
#include <dirent.h>
#include "chat.h"
#include <sys/stat.h>

#define LOGGING 100
#define SUCCESS 200
#define NO_ANSWER 500
#define MAX_CHATS 10
#define MAX_DELAY 10

FILE* f_manual = fopen("info/manual.txt", "rb");
const int m_size = sizeof(*f_manual) / sizeof(char);
char s_manual[m_size];
char mc;

static struct termios original_t;
void disableEcho() {
    struct termios t;
    tcgetattr(STDIN_FILENO, &t);
    original_t = t;
    t.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &t);
}

void enableEcho() {
    tcsetattr(STDIN_FILENO, TCSANOW, &original_t);
}

```



```

}

using namespace std;

void cli_decode_msg(char* msg, int mlen, char* output, char* request, int& status);
void cli_input(string& text);
bool client_loop(int&, int&, int&, string&, int&, char*, char*, char*, char*,
                 int&, struct timeval&, struct sockaddr_in&, struct sockaddr_in&, char*, string);
bool commandexists(string command);

int socket_init(int& sock, struct sockaddr_in* addr){
    sock = socket(AF_INET, SOCK_STREAM, 0);
    if(sock < 0){
        cout << "НЕ УДАЛОСЬ СОЗДАТЬ КЛИЕНТА!" << endl;
        return -1;
    }
    if(bind(sock, (sockaddr*)addr, sizeof(struct sockaddr_in)) < 0)
    {
        cout << "НЕ УДАЛОСЬ ИНИЦИАЛИЗИРОВАТЬ КЛИЕНТА!" << endl;
        return -1;
    }
    return sock;
}

//-----

int main()
{
    FILE* f_manual = fopen("info/manual.txt", "rb");
    char* s_manual;
    if (f_manual != NULL) {
        fseek(f_manual, 0, SEEK_END);
        long m_size = ftell(f_manual);
        fseek(f_manual, 0, SEEK_SET);
        s_manual = (char*)malloc(m_size + 1);
        size_t bytes_read = fread(s_manual, 1, m_size, f_manual);
        s_manual[bytes_read] = '\0';
        fclose(f_manual);
    }

    struct hostent *hp;

    int c_sock;
    int chat_sock;
    int lobby_sock;

    struct sockaddr_in c_addr;
    struct sockaddr_in s_addr;

    bzero((char*)&c_addr, (sizeof(struct sockaddr_in)));
    c_addr.sin_family = AF_INET;
    c_addr.sin_addr.s_addr = htonl(INADDR_ANY);
    c_addr.sin_port = 0;

    if(socket_init(c_sock, &c_addr) < 0){
        return -1;
    }
}

```

```

char g_host[BUFF_LEN] = "";
int g_port = 0;

char s_msg[BUFF_LEN] = "";
char a_msg[BUFF_LEN] = "";

int rec = 0;
char output[BUFF_LEN] = "";
char request[BUFF_LEN] = "";
int status = WAIT_ACCEPT;
string login;
string password;

struct timeval old_tv;
socklen_t len = sizeof(old_tv);
getsockopt(c_sock, SOL_SOCKET, SO_RCVTIMEO, (char*)&old_tv, &len);
struct timeval tv;
tv.tv_sec = MAX_DELAY;
tv.tv_usec = 0;
setsockopt(c_sock, SOL_SOCKET, SO_RCVTIMEO, (const char*)&tv, sizeof(tv));
//-----
cout << "===== " << endl;
cout << "                Добро пожаловать в СТАРТАП!                " << endl;
cout << "===== " << endl;

for(;;){ // ОСНОВНОЙ ЦИКЛ ЖИЗНИ КЛИЕНТА
    //=====
    // 1. ПОИСК СЕРВЕРА
    //=====
    for(;;)
    {
        bzero(s_msg, BUFF_LEN);
        bzero(a_msg, BUFF_LEN);
        bzero(output, BUFF_LEN);
        bzero(request, BUFF_LEN);
        cout << "\n    ВВЕДИТЕ АДРЕС СЕРВЕРА (или exit, чтобы выйти): ";
        cin >> g_host;
        if(strncmp(g_host, "exit", 5) == 0){
            char conf = ' ';
            cout << "=====Выход из игры!===== " <<
endl;

            do{
                cout << "                Вы уверены, что хотите выйти? (y/N): ";
                cin >> conf;
            } while (conf != 'y' && conf != 'Y' && conf != 'n' && conf != 'N');
            if(conf == 'y' || conf == 'Y'){
                cout << "=====Игра
закрывается!===== " << endl;

                return 0;
            } else {
                continue;
            }
        }
        cout << "    ВВЕДИТЕ ПОРТ СЕРВЕРА: ";
        cin >> g_port;
        cout << endl;

        hp = gethostbyname(g_host);

        bzero((char*)&s_addr, (sizeof(struct sockaddr_in)));
        s_addr.sin_family = AF_INET;
        bcopy(hp->h_addr, &s_addr.sin_addr, hp->h_length);

```

```

s_addr.sin_port = htons(g_port);

if (connect(c_sock, (sockaddr*)&s_addr, sizeof(struct sockaddr_in)) < 0) {
    cout << "    СОЕДИНЕНИЕ С СЕРВЕРОМ НЕ УДАЛОСЬ!" << endl;
    cout << endl;
    continue;
}

struct sockaddr_in local_addr;
char client_ip[INET_ADDRSTRLEN];
socklen_t len = sizeof(local_addr);
if (getsockname(c_sock, (struct sockaddr*)&local_addr, &len) == 0) {
    inet_ntop(AF_INET, &local_addr.sin_addr, client_ip, sizeof(client_ip));
}

char lr = ' ';
do{
    bzero(s_msg, BUFF_LEN);
    bzero(a_msg, BUFF_LEN);
    bzero(output, BUFF_LEN);
    bzero(request, BUFF_LEN);
    cout << "        Логин (L) /Регистрация (R) /Выйти (E) : ";
    cin >> lr;

    if(lr == 'E' || lr == 'e'){
        return 0;
    }
    int ans = -1;

    if(lr == 'R' || lr == 'r'){
        do{
            do{
                cout << "        Придумайте логин: ";
                cin >> login;
            } while(login.size() <= 0);
            string r_pas =
"fagshdgfkldafdjvixopmklfeakjsgbiodesfkagksjbfjizdpos;clfmaklgnbfxijpozks;lfmklbf";
            disableEcho();
            do{
                cout << "        Придумайте пароль: ";
                cin >> password;
                cout << endl;
                cout << "        Повторите пароль: ";
                cin >> r_pas;
                cout << endl;
                if(password != r_pas){
                    cout << "Пароли не совпадают!" << endl;
                } else {
                    break;
                }
            }while(true);
            enableEcho();
            strcat(s_msg, login.c_str());
            strcat(s_msg, ":");
            strcat(s_msg, password.c_str());
            strcat(s_msg, "|register");
            send(c_sock, s_msg, BUFF_LEN, 0);
            ans = -1;
            ans = recv(c_sock, a_msg, BUFF_LEN, 0);
            status = LOGGING;

```

```

        if(ans > 0){
            cli_decode_msg(a_msg, BUFF_LEN, output, request, status);
        }else{
            status = NO_ANSWER;
            break;
        }

        if(strncmp(request, "logbusy", 8) == 0){
            cout << "        Логин уже занят, придумайте другой..." <<

endl;

            continue;
        }
        if(strncmp(request, "success", 8) == 0){
            cout << "        Регистрация пройдена!" << endl;
            break;
        }
        if(strncmp(request, "rfail", 6) == 0){
            cout << "        Не удалось пройти регистрацию..." << endl;
            break;
        }
    } while(true);
    if(status == NO_ANSWER){
        cout << "\n        СЕРВЕР НЕ ОТВЕЧАЕТ(" << endl;
        break;
    }
    lr = ' ';
    cout << endl;
    continue;
}

cout << endl;
cout << "        Логин: ";
cin >> login;
cout << "        Пароль: ";
disableEcho();
cin >> password;
cout << endl;
enableEcho();

sprintf(s_msg, "%s:%s:%s|login",
login.c_str(), password.c_str(), client_ip);
send(c_sock, s_msg, BUFF_LEN, 0);
ans = -1;
ans = recv(c_sock, a_msg, BUFF_LEN, 0);
status = LOGGING;

if(ans > 0){
    cli_decode_msg(a_msg, BUFF_LEN, output, request, status);
}else{
    cout << "\n        СЕРВЕР НЕ ОТВЕЧАЕТ(" << endl;
    cout << endl;
    status = NO_ANSWER;
    break;
}

if(strncmp(request, "success", 8) == 0){
    cout << endl;
    status = SUCCESS;
    break;
}

if(strncmp(request, "wrong", 6) == 0){
    cout << "        Неверный логин или пароль!" << endl;

```

```

        lr = ' ';
        cout << endl;
        continue;
    }

    if(strncmp(request, "online", 7) == 0){
        cout << "    Пользователь уже онлайн!" << endl;
        lr = ' ';
        cout << endl;
        continue;
    }

    }while(lr != 'L' && lr != 'l' && lr != 'R' && lr != 'r' && lr != 'E' && lr !=
'e');

    if(status == SUCCESS){
        break;
    } else {
        close(c_sock);
        if(socket_init(c_sock, &c_addr) < 0){
            cout << "    Не удалось реинициализировать клиента." << endl;
            return -1;
        }
        continue;
    }
}

string chats_path = "info/chats/" + login + "/";
mkdir(chats_path.c_str(), 0777);

cout << "                Вы успешно вошли на сервер!" << endl;
cout << "    ПОДСКАЗКА: для просмотра доступных команд введите help" << endl;

//=====
// ОТКРЫВАЕМ ПРОСЛУШКУ ПОЛЬЗОВАТЕЛЮ
//=====
int chat_server_sock;
struct sockaddr_in chat_addr;
bzero(&chat_addr, sizeof(chat_addr));
chat_addr.sin_family = AF_INET;
chat_addr.sin_addr.s_addr = htonl(INADDR_ANY);
chat_addr.sin_port = 0;
if (socket_init(chat_server_sock, &chat_addr) < 0) {
    cout << "Не удалось создать сокет для приёма сообщений" << endl;
} else {
    socklen_t addr_len = sizeof(chat_addr);
    getsockname(chat_server_sock, (sockaddr*)&chat_addr, &addr_len);
    int chat_port = ntohs(chat_addr.sin_port);

    bzero(s_msg, BUFF_LEN);
    sprintf(s_msg, "%s:%d|setchatport", login.c_str(), chat_port);
    send(c_sock, s_msg, BUFF_LEN, 0);
    if(recv(c_sock, a_msg, BUFF_LEN, 0) < 0 || strcmp(request, "success", 8) !=
0){
        cout << "Не удалось обеспечить приём сообщений (server error)" << endl;
    }else{
        listen(chat_server_sock, MAX_CHATS);

        chat_args* cargs = new chat_args();
        cargs->socket = chat_server_sock;
        cargs->login = login;

        pthread_t accept_tid;

```

```

        pthread_create(&accept_tid, NULL, msg_accept_thread, (void*)cargs);
        pthread_detach(accept_tid);

        // cout << "Приём сообщений запущен на порту " << chat_port << endl;
    }
}
//=====

while(client_loop(c_sock, chat_sock, lobby_sock, login, rec,
    s_msg, a_msg, output, request, status, old_tv, s_addr, c_addr, s_manual,
    chats_path)); // ЦИКЛ СЕССИИ НА СЕРВЕРЕ
close(chat_server_sock);
close(c_sock);
if(socket_init(c_sock, &c_addr) < 0){
    cout << "    Не удалось реинициализировать клиента." << endl;
    return -1;
}
}
free(s_manual);
close(c_sock);
return 0;
}

bool client_loop(int& c_sock, int& chat_sock, int& lobby_sock, string& login, int& rec,
    char* s_msg, char* a_msg, char* output, char* request, int& status, struct timeval&
    old_tv,
    struct sockaddr_in& s_addr, struct sockaddr_in& c_addr, char* manual, string
    chats_path)
{
    //=====
    // 2. Командная строка клиента для взаимодействия с сервером
    //=====
    for(;;)
    {
        bzero(s_msg, BUFF_LEN);
        bzero(a_msg, BUFF_LEN);
        bzero(output, BUFF_LEN);
        bzero(request, BUFF_LEN);
        cout << endl;
        string command;
        int ans = -1;
        cout << "[" << login << "]"$ ";
        cin >> command;
        if(!commandexists(command)){
            cout << "Неизвестная команда '" << command << "'" << endl;
            cout << "ПОДСКАЗКА: для просмотра доступных команд введите help" << endl;
            continue;
        }

        //-----
        // Инструкция
        //-----
        if(strncmp(command.c_str(), "help", 5) == 0){
            cout << endl;
            cout << "'ps' - показать список игроков онлайн." << endl;
            cout << "'psa' - показать список всех игроков." << endl;
            cout << "'rt' - показать рейтинг." << endl;
            cout << "'ls' - показать список лобби." << endl;
            cout << "'mkl' - создать лобби." << endl;
            cout << "'join' - присоединиться к лобби." << endl;
            cout << "'chats' - показать чаты." << endl;
            cout << "'chat' - показать чат с игроком." << endl;
            cout << "'mes' - написать сообщение игроку." << endl;
        }
    }
}

```

```

        cout << "'clchat' - очистить чат с игроком." << endl;
        cout << "'about' - об игре." << endl;
        cout << "'exit' - выйти." << endl;

        //cout << "'rm chat [id игрока]' - удалить чат с игроком" << endl; --- В
ДОЛГИЙ ЯЩИК
        //cout << "'report [id игрока]' - отправить жалобу на игрока." << endl; --- В
ДОЛГИЙ ЯЩИК
        continue;
    }

    //-----
    // Запрос списка игроков
    //-----
    if(strncmp(command.c_str(), "psa", 6) == 0){
        strcat(s_msg, "|getallplayers");
        send(c_sock, s_msg, BUFF_LEN, 0);
        ans = -1;
        ans = recv(c_sock, a_msg, BUFF_LEN, 0);
        cli_decode_msg(a_msg, BUFF_LEN, output, request, status);
        if(ans > 0){
            cout << output << endl;
            continue;
        }
    }

    if(strncmp(command.c_str(), "ps", 2) == 0){
        strcat(s_msg, "|getplayers");
        send(c_sock, s_msg, BUFF_LEN, 0);
        ans = -1;
        ans = recv(c_sock, a_msg, BUFF_LEN, 0);
        cli_decode_msg(a_msg, BUFF_LEN, output, request, status);
        if(ans > 0){
            cout << output << endl;
            continue;
        }
    }

    //-----
    // Запрос рейтинга
    //-----
    if(strncmp(command.c_str(), "rt", 3) == 0){
        strcat(s_msg, "|rate");
        send(c_sock, s_msg, BUFF_LEN, 0);
        ans = -1;
        ans = recv(c_sock, a_msg, BUFF_LEN, 0);
        cli_decode_msg(a_msg, BUFF_LEN, output, request, status);
        if(ans > 0){
            cout << output << endl;
            continue;
        }
    }

    //-----
    // Запрос списка лобби
    //-----
    if(strncmp(command.c_str(), "ls", 3) == 0){
        strcat(s_msg, "|getlobby");
        send(c_sock, s_msg, BUFF_LEN, 0);
        ans = -1;
        ans = recv(c_sock, a_msg, BUFF_LEN, 0);
        cli_decode_msg(a_msg, BUFF_LEN, output, request, status);
        if(ans > 0){

```

```

        cout << output << endl;
        continue;
    }
}

//-----
// Запрос на создание лобби
//-----
if(strncmp(command.c_str(), "mkl", 5) == 0){
    string name;
    cout << "Введите название лобби (или exit для отмены): ";
    cin >> name;
    if(strncmp(name.c_str(), "exit", 5) == 0){
        continue;
    }
    int num = -1;
    do{
        cout << "Введите количество игроков (3-6, или 0 для отмены): ";
        cin >> num;
    } while((num < 3 || num > 6) && num != 0);
    if(num == 0){
        continue;
    }
    sprintf(s_msg, "%d:", num);
    strcat(s_msg, name.c_str());
    strcat(s_msg, "|makelob");
    send(c_sock, s_msg, BUFF_LEN, 0);
    ans = -1;
    ans = recv(c_sock, a_msg, BUFF_LEN, 0);
    cli_decode_msg(a_msg, BUFF_LEN, output, request, status);
    if(ans > 0){
        if(strncmp(request, "success", 8) == 0){
            cout << "Ваше лобби '" << name << "' успешно создано!" << endl;
        }else{
            cout << "Не удалось создать лобби." << endl;
        }
        continue;
    }
}

//-----
// Запрос на присоединение к лобби
//-----
if(strncmp(command.c_str(), "join", 5) == 0){
    int lobby_id;
    cout << "    Введите ID лобби: ";
    cin >> lobby_id;

    sprintf(s_msg, "%d", lobby_id);
    strcat(s_msg, "|join");
    send(c_sock, s_msg, BUFF_LEN, 0);
    bzero(s_msg, BUFF_LEN);
    ans = -1;
    ans = recv(c_sock, a_msg, BUFF_LEN, 0);
    if(ans > 0){
        cli_decode_msg(a_msg, BUFF_LEN, output, request, status);
        if(strncmp(request, "allow", 6) == 0){
            if(socket_init(lobby_sock, &c_addr) < 0){
                cout << "Не удалось подключиться к лобби. (sock error)" << endl;
                continue;
            }
            int lobby_port = atoi(output);
            sockaddr_in lobby_addr;

```



```

        bzero(&lobby_addr, sizeof(struct sockaddr_in));
        bcopy(&s_addr, &lobby_addr, sizeof(struct sockaddr_in));
        lobby_addr.sin_family = AF_INET;
        lobby_addr.sin_port = htons(lobby_port);
        if(connect(lobby_sock, (sockaddr*)&lobby_addr, sizeof(struct
sockaddr_in)) < 0){
            cout << "Не удалось подключиться к лобби." << endl;
            continue;
        }
        status = WAIT_ACCEPT;
        strcat(s_msg, login.c_str());
        strcat(s_msg, "|join");
        send(lobby_sock, s_msg, BUFF_LEN, 0);
        break;
    } else {
        cout << "Не удалось найти указанное лобби." << endl;
        continue;
    }
    continue;
}

}

//-----
// Запрос списка чатов
//-----
if(strncmp(command.c_str(), "chats", 6) == 0){
    DIR* dir = opendir(chats_path.c_str());
    if(dir == NULL){
        cout << "Папка с чатами не найдена: " << chats_path << endl;
        continue;
    }

    vector<string> chats;
    struct dirent* entry;

    while((entry = readdir(dir)) != NULL){
        string filename = entry->d_name;
        if(filename.length() > 4 && filename.substr(filename.length() - 4) ==
".txt"){
            string chat_name = filename.substr(0, filename.length() - 4);
            chats.push_back(chat_name);
        }
    }
    closedir(dir);

    if(chats.empty()){
        cout << "Нет активных чатов" << endl;
    } else {
        cout << "Активные чаты:" << endl;
        for(size_t i = 0; i < chats.size(); i++){
            cout << "    " << i+1 << ") " << chats[i] << endl;
        }
    }
    continue;
}

//-----
// Запрос чата с игроком
//-----
if(strncmp(command.c_str(), "chat", 5) == 0){
    string nick;
    cout << "Введите ник игрока: ";

```

```

cin >> nick;
string chatname = chats_path + nick + ".txt";
FILE* f_chat = fopen(chatname.c_str(), "r");
char* s_chat;
if(f_chat == NULL){
    cout << "Чат не найден..." << endl;
    continue;
}
fseek(f_chat, 0, SEEK_END);
long m_size = ftell(f_chat);
fseek(f_chat, 0, SEEK_SET);
s_chat = (char*)malloc(m_size + 1);
size_t bytes_read = fread(s_chat, 1, m_size, f_chat);
s_chat[bytes_read] = '\0';
fclose(f_chat);
cout << "Чат с " << nick << ":" << endl;
if(m_size <= 0){
    cout << "Чат пуст..." << endl;
    continue;
}
cout << s_chat << endl;
continue;
}

//-----
// Запрос очистки чата с игроком
//-----
if(strncmp(command.c_str(), "clchat", 7) == 0){
    string nick;
    cout << "Введите ник игрока: ";
    cin >> nick;
    string chatname = chats_path + nick + ".txt";
    FILE* f_chat = fopen(chatname.c_str(), "w");
    if(f_chat == NULL){
        cout << "Чат не найден..." << endl;
        continue;
    }
    cout << "Чат с игроком " << nick << " очищен..." << endl;
    continue;
}

if(strncmp(command.c_str(), "mes", 4) == 0){
    string t_nick;
    cout << "Введите ник игрока: ";
    cin >> t_nick;
    string chatname = chats_path + t_nick + ".txt";
    strcat(s_msg, t_nick.c_str());
    strcat(s_msg, "|finduser");
    send(c_sock, s_msg, BUFF_LEN, 0);
    ans = -1;
    ans = recv(c_sock, a_msg, BUFF_LEN, 0);
    cli_decode_msg(a_msg, BUFF_LEN, output, request, status);
    if(ans > 0){
        if(strncmp(request, "nop", 4) == 0){
            cout << "Игрок " << t_nick << " не найден." << endl;
            continue;
        }
        if(strncmp(request, "off", 4) == 0){
            cout << "Игрок " << t_nick << " не в сети." << endl;
            continue;
        }
    }

    close(chat_sock);

```

```

        if(socket_init(chat_sock, &c_addr) < 0){
            cout << "Не удалось инициализировать отправку сообщения." << endl;
            continue;
        }

        char ip_char[BUFF_LEN] = "";
        int port = 0;

        sscanf(output, "%[^:]:%d", ip_char, &port);

        struct sockaddr_in t_addr;
        bzero(&t_addr, sizeof(struct sockaddr_in));
        t_addr.sin_family = AF_INET;
        inet_aton(ip_char, &t_addr.sin_addr);
        t_addr.sin_port = htons(port);

        if(connect(chat_sock, (sockaddr*)&t_addr, sizeof(sockaddr_in)) < 0){
            cout << "Не удалось установить соединение с " << t_nick << "." <<
endl;
            continue;
        }
        bzero(s_msg, BUFF_LEN);
        char ts[BUFF_LEN];
        string mes;
        cout << "Введите ваше сообщение:" << endl;
        cin.ignore();
        cli_input(mes);
        strcat(ts, login.c_str());
        strcat(ts, ": ");
        strcat(ts, mes.c_str());
        strcat(ts, "\n");
        strcat(s_msg, ts);
        strcat(s_msg, "|sent");
        send(chat_sock, s_msg, BUFF_LEN, 0);
        recv(chat_sock, a_msg, BUFF_LEN, 0);
        cli_decode_msg(a_msg, BUFF_LEN, output, request, status);
        if(strncmp(request, "received", 9) == 0){
            cout << "Сообщение отправлено!" << endl;
            FILE* f_chat = fopen(chatname.c_str(), "a");
            if(f_chat == NULL){
                cout << "Чат не найден..." << endl;
                bzero(ts, BUFF_LEN);
                continue;
            }
            fprintf(f_chat, "%s", ts);
            fclose(f_chat);
        }else{
            cout << "Не удалось доставить сообщение! Попробуйте ещё раз." << endl;
        }
        bzero(ts, BUFF_LEN);
        continue;
    }
}

//-----
//Об игре
//-----
if(strncmp(command.c_str(), "about", 6) == 0){
    cout << manual << endl;
    continue;
}

```

```

//-----
// Выход из игры
//-----

if(strncmp(command.c_str(), "exit", 5) == 0){
    char conf = ' ';
    cout << "=====Выход из игры!===== " <<
endl;
    do{
        cout << "          Вы уверены, что хотите выйти? (y/N): ";
        cin >> conf;
    } while (conf != 'y' && conf != 'Y' && conf != 'n' && conf != 'N');
    if(conf == 'y' || conf == 'Y'){
        cout << "=====Выход с сервера!===== "
<< endl;

        strcat(s_msg, "|exit");
        send(c_sock, s_msg, BUFF_LEN, 0);
        return false;
    } else {
        continue;
    }
}

if(ans <= 0)
{
    cout << "\nСЕРВЕР НЕ ОТВЕЧАЕТ(" << endl;
    continue;
}

}

//=====
// 3. Игра
//=====
for(;;)
{
    bzero(s_msg, BUFF_LEN);
    bzero(a_msg, BUFF_LEN);
    bzero(output, BUFF_LEN);
    bzero(request, BUFF_LEN);
    rec = recv(lobby_sock, a_msg, BUFF_LEN, 0);
    if (rec == 0)
    {
        cout << "СОЕДИНЕНИЕ ПРЕРВАНО!" << endl;
        close(lobby_sock);
        return false;
    }
    if (rec < 0)
    {
        cout << " ОШИБКА СЕРВЕРА! (Invalid server socket)" << endl;
        close(lobby_sock);
        return false;
    }
    cli_decode_msg(a_msg, BUFF_LEN, output, request, status);

    if(strncmp(request, "over", 5) == 0){
        cout << output << endl;
        sleep(3);
        close(lobby_sock);
        return true; // возвращение в командную строку
    }
}

```

```

if(strncmp(request, "common", 7) == 0){
    cout << output << endl;
    send(lobby_sock, "EmptyMes", 2, 0);
    continue;
}

if(status == WAIT_ACCEPT)
{
    //continue;
}
if(status == PRE_TO_PLAY){
    if(strncmp(request, "getready", 9) == 0){
        char a;
        cout << "    Вы успешно присоединились к игре!" << endl;
        cout << "    Готовы начать игру?" << endl;
        do{
            cout << "    Введите Y(Готов)/q(Выйти): ";
            cin >> a;
        } while (a != 'y' && a != 'Y' && a != 'q' && a != 'Q');
        if (a == 'q' || a == 'Q'){
            close(lobby_sock);
            socket_init(lobby_sock, &c_addr);
            return true;
        }
        if(send(lobby_sock, "|readytoplay", BUFF_LEN, 0) < 0){
            cout << "    Не удалось отправить сообщение. Попробуйте ещё раз." <<
endl;

        } else {
            cout << "    Ожидание других игроков..." << endl;
            cout << endl;
        }
        continue;
    }
    continue;
}
if(status == WAITING || status == READY_TO_PLAY){
    if(output[0] != ' ' && output[0] != '\\0'){
        cout << output << endl;
    }
    continue;
}
if(status == EMPLOYER){
    if(strncmp(request, "givehist", 9) == 0){
        string manual;
        cout << endl;
        output[0] = ' ';
        cout << output << endl;
        cin.ignore();
        cli_input(manual);

        strcat(s_msg, manual.c_str());
        strcat(s_msg, "|sendhist");
        send(lobby_sock, s_msg, BUFF_LEN, 0);
        cout << endl;
        continue;
    }
    if(strncmp(request, "givejchoice", 11) == 0){
        cout << endl;
        cout << output << endl;

        string choice;
        cout << "    Введите ваш выбор: ";
        cin.ignore();

```

```

        getline(cin, choice);

        bzero(s_msg, BUFF_LEN);
        strcat(s_msg, choice.c_str());
        strcat(s_msg, "|jchoice");
        send(lobby_sock, s_msg, BUFF_LEN, 0);
        continue;
    }
    continue;
}

if(status == ANSWERING){
    if(strncmp(request, "areanswerm", 11) == 0){
        cout << "    Ваша очередь на собеседование!" << endl;
        cout << output << endl;
        int v = 0;
        do{
            cout << "    Введите номер вакансии, на которую вы претендуете (1 - 3):
";

            cin >> v;
        } while(v != 1 && v != 2 && v != 3);
        sprintf(s_msg, "%d", v);
        strcat(s_msg, "|claim");
        send(lobby_sock, s_msg, BUFF_LEN, 0);
        continue;
    }

    if(strncmp(request, "claim", 6) == 0){
        char r = ' ';
        do{
            cout << "\n    Работодатель готов Вас выслушать. Введите 'Y', когда
будете готовы отвечать на собеседовании: ";
            cin >> r;
        } while(r != 'Y' && r != 'y');
        send(lobby_sock, "|readytoanswer", BUFF_LEN, 0);
        continue;
    }

    if(strncmp(request, "giveanswerm", 11) == 0){
        string resume;
        cout << "    Минута пошла! Удачи!" << endl;
        cin.ignore();
        cli_input(resume);
        strcat(s_msg, resume.c_str());
        strcat(s_msg, "|sendanswer");
        send(lobby_sock, s_msg, BUFF_LEN, 0);
        continue;
    }

    if(strncmp(request, "quest", 6) == 0){
        string answ;
        cout << output << endl;
        cout << endl;
        cout << "    Введите ваш ответ." << endl;
        cli_input(answ);
        strcat(s_msg, answ.c_str());
        strcat(s_msg, "|aquest");
        send(lobby_sock, s_msg, BUFF_LEN, 0);
        continue;
    }

    if(strncmp(request, "print", 6) == 0){
        cout << output << endl;
    }
}

```

```

        strcat(s_msg, "|hquest");
        send(lobby_sock, s_msg, BUFF_LEN, 0);
        continue;
    }

    if(status == QUESTIONING){
        if(strncmp(request, "gquest", 7) == 0){
            cout << "\n Ваш вопрос отправлен и дойдёт до адресата в порядке очереди."
<< endl;

            send(lobby_sock, " ", 2, 0);
            continue;
        }
        char h = ' ';
        do{
            cout << "\n Есть ли у Вас вопросы к соискателю? (Y/n): ";
            cin >> h;
        } while(h != 'Y' && h != 'y' && h != 'N' && h != 'n');
        if(h == 'N' || h == 'n'){
            send(lobby_sock, "|noquest", BUFF_LEN, 0);
            cout << " Ожидайте заверения секции с вопросами..." << endl;
            continue;
        }
        string question;
        cout << " Напишите Ваш вопрос." << endl;
        cin.ignore();
        cli_input(question);
        strcat(s_msg, output);
        strcat(s_msg, question.c_str());
        strcat(s_msg, "|quest");
        send(lobby_sock, s_msg, BUFF_LEN, 0);
        continue;
    }

    if(status == SCORING){
        int score = 0;
        if(strncmp(request, "print", 6) == 0){
            cout << output << endl;
            send(lobby_sock, " ", 2, 0);
            continue;
        }
        if(strncmp(request, "score", 6) == 0){
            do{
                cout << "\n Поставьте выступившему оценку от 1 до 5: ";
                cin >> score;
            } while(score < 1 || score > 5);
            sprintf(s_msg, "%d", score);
            strcat(s_msg, "|score");
            send(lobby_sock, s_msg, BUFF_LEN, 0);
            cout << " Оценка отправлена!" << endl;
            continue;
        }
        send(lobby_sock, " ", 2, 0);
        continue;
    }
}

return true;
}

// Структура сообщения (сервер -> клиент) "output|request|p_status"
void cli_decode_msg(char* msg, int mlen, char* output, char* request, int& status){
    bzero(output, mlen);
    bzero(request, mlen);

```

```

int bc = get_line_b(output, msg, 0, mlen, '|');
bc = get_line_b(request, msg, bc, mlen, '|');
char pstat[BUFF_LEN] = "";
bc = get_line_b(pstat, msg, bc, mlen, '|');
if (strncmp(pstat, "WAIT_ACCEPT", 12) == 0){
    status = WAIT_ACCEPT;
}
if (strncmp(pstat, "PRE_TO_PLAY", 12) == 0){
    status = PRE_TO_PLAY;
}
if(strncmp(pstat, "READY_TO_PLAY", 14) == 0){
    status = READY_TO_PLAY;
}
if(strncmp(pstat, "WAITING", 8) == 0){
    status = WAITING;
}
if(strncmp(pstat, "EMPLOYER", 9) == 0){
    status = EMPLOYER;
}
if(strncmp(pstat, "ANSWERING", 10) == 0){
    status = ANSWERING;
}
if(strncmp(pstat, "QUESTIONING", 12) == 0){
    status = QUESTIONING;
}
if(strncmp(pstat, "SCORING", 8) == 0){
    status = SCORING;
}
if(strncmp(pstat, "LEFT", 5) == 0){
    status = LEFT;
}
if(strncmp(pstat, "SUCCESS", 8) == 0){
    status = SUCCESS;
}
}

void cli_input(string& text){
    cout << "    (для завершения введите пустую строку или 'END'):\n" << endl;
    //cin.ignore();
    text = " ";
    long unsigned int max_len = OUT_LEN;
    string line;
    while (true) {
        cout << "(" << text.size() - 1 << "/" << max_len << "): ";
        getline(cin, line);
        if(text.size() + line.size() > max_len){
            cout << "    Не хватает места!" << endl;
            continue;
        }
        if (line.empty() || line == "END") {
            break;
        }
        text += "    " + line + "\n";
    }

    if (!text.empty() && text.back() == '\n') {
        text.pop_back();
    }
}

bool commandexists(string command){
    return command == "ps" || command == "psa" || command == "rt" || command == "exit" ||
    command == "ls"

```



```

        || command == "mkl" || command == "join" || command == "chat" || command ==
"chats" || command == "help"
        || command == "about" || command == "clchat" || command == "mes";
}

```

#### server.cpp

```

// СЕРВЕР (ИСХОДНЫЙ КОД)
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <time.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <pthread.h>
#include <cstring>
#include "lobby.h"
#include <termios.h>

#define MAX_USERS 100

using namespace std;

static struct termios original_t;

StartupDbContext* CONTEXT;

void disableEcho() {
    struct termios t;
    tcgetattr(STDIN_FILENO, &t);
    original_t = t;
    t.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &t);
}

void enableEcho() {
    tcsetattr(STDIN_FILENO, TCSANOW, &original_t);
}

```

```

}

void* user_thread(void* arg)
{
    int socket = (int)(long)arg;
    char s_msg[BUFF_LEN] = "";
    char a_msg[BUFF_LEN] = "";
    //-----
    char output[BUFF_LEN] = "";
    char request[BUFF_LEN] = "";
    //-----
    int rec_l = 0;
    string p_login;
    // Обработка запросов пользователей
    for(;;)
    {
        bzero(s_msg, BUFF_LEN);
        bzero(a_msg, BUFF_LEN);
        rec_l = recv(socket, a_msg, BUFF_LEN, 0);
        if(rec_l <= 0){
            CONTEXT->logout(p_login);
            close(socket);
            pthread_exit(0);
        }
        ser_decode_msg(a_msg, BUFF_LEN, output, request);

        if(strncmp(request, "login", 6) == 0){
            char login[BUFF_LEN] = "";
            char password[BUFF_LEN] = "";
            char ip[BUFF_LEN] = "";
            sscanf(output, "%[^:]:%[^:]:%[^:]", login, password, ip);

            int lstat = CONTEXT->auth(string(login), string(password), string(ip));

            if(lstat == L_WRONG){
                send(socket, "|wrong|", BUFF_LEN, 0);
            } else if (lstat == L_ONLINE){
                send(socket, "|online|", BUFF_LEN, 0);
            } else if (lstat == L_SUCCESS){
                p_login = login;
                send(socket, "|success|", BUFF_LEN, 0);
            }
            continue;
        }

        if(strncmp(request, "register", 9) == 0){
            string login_pass = output;
            size_t separator = login_pass.find(':');

            string login = login_pass.substr(0, separator);
            string password = login_pass.substr(separator + 1);

            char hashed[crypto_pwhash_STRBYTES];
            if(crypto_pwhash_str(hashed, password.c_str(), password.length(),
                                crypto_pwhash_OPSLIMIT_INTERACTIVE,
                                crypto_pwhash_MEMLIMIT_INTERACTIVE) < 0){
                cout << "ОШИБКА ШИФРОВАНИЯ! (" << login << ")" << endl;
            }

            int rstat = CONTEXT->reg(login, hashed);
            if(rstat == R_BUSY){
                send(socket, "|logbusy|", BUFF_LEN, 0);
            }
        }
    }
}

```

```

    }else if (rstat == R_FAIL){
        send(socket, "|rfail|", BUFF_LEN, 0);
    }else if (rstat == R_SUCCESS){
        send(socket, "|success|", BUFF_LEN, 0);
    }
    continue;
}

if(strncmp(request, "getplayers", 11) == 0){
    strcat(s_msg, CONTEXT->get_players_on().c_str());
    strcat(s_msg, "|");
    send(socket, s_msg, BUFF_LEN, 0);
    continue;
}

if(strncmp(request, "getallplayers", 14) == 0){
    strcat(s_msg, CONTEXT->get_players_all().c_str());
    strcat(s_msg, "|");
    send(socket, s_msg, BUFF_LEN, 0);
    continue;
}

if(strncmp(request, "rate", 5) == 0){
    strcat(s_msg, CONTEXT->get_rating().c_str());
    strcat(s_msg, "|");
    send(socket, s_msg, BUFF_LEN, 0);
    continue;
}

if(strncmp(request, "makelob", 8) == 0){
    char name[BUFF_LEN] = "";
    int num = 0;
    sscanf(output, "%d:%[^|]", &num, name);
    if(num < 3 || num > 6){
        send(socket, "|NO", BUFF_LEN, 0);
        continue;
    }
    int lob_id = CONTEXT->add_lobby(p_login, name, num);
    if(lob_id >= 0){
        send(socket, "|success", BUFF_LEN, 0);
        pthread_t lobby_t;
        lobby_args largs;
        largs.id = lob_id;
        largs.context = CONTEXT;
        largs.size = num;
        pthread_create(&lobby_t, NULL, lobby_thread, (void*)&largs);
        pthread_detach(lobby_t);
        sleep(1);
    } else {
        send(socket, "|NO", BUFF_LEN, 0);
    }
    continue;
}

if(strncmp(request, "join", 5) == 0){
    int id = atoi(output);
    int port = CONTEXT->join_lobby(id);
    sprintf(s_msg, "%d", port);
    if(port > 0){
        strcat(s_msg, "|allow");
    }else{
        strcat(s_msg, "|NO");
    }
}

```

```

    }
    send(socket, s_msg, BUFF_LEN, 0);
    continue;
}

if(strncmp(request, "getlobby", 9) == 0){
    strcat(s_msg, CONTEXT->get_lobbies().c_str());
    strcat(s_msg, "|");
    send(socket, s_msg, BUFF_LEN, 0);
    continue;
}

if(strncmp(request, "finduser", 9) == 0){
    string nick = (string)output;
    string ip;
    bool online;
    int port;
    if(!CONTEXT->finduser(nick, ip, port, online)){
        strcat(s_msg, "|nop");
        send(socket, s_msg, BUFF_LEN, 0);
        continue;
    }
    if(!online){
        strcat(s_msg, "|off");
        send(socket, s_msg, BUFF_LEN, 0);
        continue;
    }
    sprintf(s_msg, "%s:%d|found", ip.c_str(), port);
    send(socket, s_msg, BUFF_LEN, 0);
    continue;
}

if(strncmp(request, "setchatport", 12) == 0){
    char login[BUFF_LEN];
    int port;
    sscanf(output, "%[^:]:%d", login, &port);
    if(CONTEXT->setuport(login, port)){
        send(socket, "success", BUFF_LEN, 0);
    }else{
        send(socket, "fail", BUFF_LEN, 0);
    }
    continue;
}

if(strncmp(request, "exit", 9) == 0){
    CONTEXT->logout(p_login);
    break;
}

// чат с игроком открывается в обход сервера в отдельном соединении между
игроками.

    send(socket, " ", 2, 0); // пустое сообщение, чтобы если что, клиент не застрял
}
close(socket);
pthread_exit(0);
}

int main()
{
    srand(time(NULL));
    pid_t server_id = getpid();
    cout << "    ID сервера: " << server_id << endl;

```

```

struct sockaddr_in s_addr;

int ser_socket = socket(AF_INET, SOCK_STREAM, 0);

if (ser_socket < 0) {
    cout << "    ОШИБКА: НЕ УДАЛОСЬ ЗАПУСТИТЬ СЕРВЕР!" << endl;
    return -1;
}

bzero((char*)&s_addr, sizeof(struct sockaddr_in));
s_addr.sin_family = AF_INET;
s_addr.sin_addr.s_addr = htonl(INADDR_ANY);
s_addr.sin_port = 0;

if(bind(ser_socket, (sockaddr*)&s_addr, sizeof(struct sockaddr_in)) < 0){
    cout << "    ОШИБКА: НЕ УДАЛОСЬ ИНИЦИАЛИЗИРОВАТЬ СЕРВЕР!" << endl;
    return -1;
}
unsigned int s_len = sizeof(struct sockaddr_in);
if (getsockname(ser_socket, (struct sockaddr*)&s_addr, &s_len) < 0){
    cout << "    ОШИБКА: НЕ УДАЛОСЬ НАЙТИ ПОРТ СЕРВЕРА!" << endl;
    return -1;
}
cout << "    АДРЕС СЕРВЕРА: " << inet_ntoa(s_addr.sin_addr) << endl;
cout << "    ПОРТ СЕРВЕРА: " << ntohs(s_addr.sin_port) << endl;
cout << endl;
cout << "Необходимо подключиться к базе данных." << endl;
string admin;
string pswrd;
cout << "Data login: ";
cin >> admin;
cout << "Data password: ";
disableEcho();
cin >> pswrd;
cout << endl;
enableEcho();

CONTEXT = new StartupDbContext(S_ADDRESS, S_PORT, DATABASE, admin, pswrd);
CONTEXT->clear_online();
CONTEXT->clear_lobbies();
if(listen(ser_socket, MAX_USERS) < 0){
    cout << "    ОШИБКА: НЕ УДАЛОСЬ ОТКРЫТЬ СЕРВЕР!" << endl;
    return -1;
}
cout << "СЕРВЕР ГОТОВ К РАБОТЕ!" << endl;
// Приём пользователей
for(;;)
{
    int usr_socket = 0;
    usr_socket = accept(ser_socket, 0, 0);
    pthread_t thread_id;
    pthread_create(&thread_id, NULL, user_thread, (void*)(long)usr_socket);
    pthread_detach(thread_id);
}
CONTEXT->clear_online();
CONTEXT->clear_lobbies();
close(ser_socket);
return 0;
}

```

## 5.2. Библиотека “Lobby”

### lobby.h

```
// СТАРТАП-ЛОББИ (БИБЛИОТЕКА)
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <time.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <pthread.h>
#include "game.h"
#include <cstring>

using namespace std;

typedef struct player_args {
    int socket;
    Game* game;
    vector<int*>* subs;
    int lobby_id;
    StartupDbContext* context;
    pthread_mutex_t* mutex;
} player_args;

typedef struct lobby_args {
    int id;
    StartupDbContext* context;
    int size;
} lobby_args;

void ser_decode_msg(char* msg, int mlen, char* output, char* request);
void send_to_all(vector<int*>*, char*, int);
void send_to_all_expt(vector<int*>*, int, char*, int);
void* player_thread(void* arg);
void* lobby_thread(void* arg);
```

### lobby.cpp

```

// СТАРТАП-ЛОББИ (ИСХОДНЫЙ КОД)

#include "lobby.h"

void* player_thread(void* arg)
{
    player_args* pargs = (player_args*)arg;
    int socket = pargs->socket;
    Game* GAME = pargs->game;
    vector<int>* SUBS = pargs->subs;
    int lobby_id = pargs->lobby_id;
    StartupDbContext* CONTEXT = pargs->context;
    pthread_mutex_t* mutex = pargs->mutex;

    int id = 0;
    char s_msg[BUFF_LEN] = "";
    char a_msg[BUFF_LEN] = "";
    //-----
    char output[BUFF_LEN] = "";
    char request[BUFF_LEN] = "";
    //-----
    int p_status = WAIT_ACCEPT;
    int rec_l = 0;
    int g_status = GAME->getStatus();
    if (recv(socket, a_msg, BUFF_LEN, 0) <= 0){
        strcat(s_msg, "      Неудачная попытка подключения|common|");
        pthread_mutex_lock(mutex);
        send_to_all(SUBS, s_msg, BUFF_LEN);
        pthread_mutex_unlock(mutex);
        close(socket);
        pthread_exit(0);
    }
    ser_decode_msg(a_msg, BUFF_LEN, output, request);
    if (strncmp(request, "join", 4) == 0){
        strcat(s_msg, GAME->addPlayer(output, socket).c_str());
        strcat(s_msg, "|common|");
        pthread_mutex_lock(mutex);
        send_to_all(SUBS, s_msg, BUFF_LEN);
        pthread_mutex_unlock(mutex);
        bzero(s_msg, BUFF_LEN);
        id = GAME->get_player_id(output);
        if(id < 0){
            strcat(s_msg, "      ОШИБКА ID.|common|");
            pthread_mutex_lock(mutex);
            send_to_all(SUBS, s_msg, BUFF_LEN);
            pthread_mutex_unlock(mutex);
            close(socket);
            pthread_exit(0);
        }
        pthread_mutex_lock(mutex);
        CONTEXT->set_lobby_num(lobby_id, GAME->getPnum());
        strcat(s_msg, "|getready|PRE_TO_PLAY");
        send(socket, s_msg, BUFF_LEN, 0);
        GAME->set_player_status(id, PRE_TO_PLAY);
        pthread_mutex_unlock(mutex);
    }
    for(;;){
        bzero(s_msg, BUFF_LEN);
        bzero(a_msg, BUFF_LEN);
        bzero(output, BUFF_LEN);
        bzero(request, BUFF_LEN);
        if(GAME->getStatus() == OVER){
            break;

```

```

}
rec_l = recv(socket, a_msg, BUFF_LEN, 0);
p_status = GAME->get_player_status(id);
g_status = GAME->getStatus();
if (rec_l == 0){
    pthread_mutex_lock(mutex);
    strcat(s_msg, GAME->remPlayer(id).c_str());
    strcat(s_msg, "|common|");
    CONTEXT->set_lobby_num(lobby_id, GAME->getPnum());
    for(vector<int>::iterator itd = SUBS->begin(); itd != SUBS->end(); itd++){
        if(*itd == id){
            SUBS->erase(itd);
            break;
        }
    }
    send_to_all(SUBS, s_msg, BUFF_LEN);
    pthread_mutex_unlock(mutex);
    break;
}
if (rec_l < 0){
    sprintf(s_msg, "    Разрыв соединения с игроком %s из-за ошибки
сокета.|common|", GAME->get_player_nick(id).c_str());
    pthread_mutex_lock(mutex);
    GAME->remPlayer(id);
    for(vector<int>::iterator itd = SUBS->begin(); itd != SUBS->end(); itd++){
        if(*itd == id){
            SUBS->erase(itd);
            break;
        }
    }
    pthread_mutex_unlock(mutex);
    break;
}
ser_decode_msg(a_msg, BUFF_LEN, output, request);

if(p_status == PRE_TO_PLAY){
    if(strncmp(request, "readytoplay", 12) == 0){
        pthread_mutex_lock(mutex);
        SUBS->push_back(socket);
        GAME->set_player_status(id, READY_TO_PLAY);
        sprintf(s_msg, "    %s готов играть!\n    Готовы: %d / %d\n|common|", GAME-
>get_player_nick(id).c_str(),
                                GAME->getRnum(), GAME->getMnum());
        send_to_all(SUBS, s_msg, BUFF_LEN);
        pthread_mutex_unlock(mutex);
        bzero(s_msg, BUFF_LEN);
        strcat(s_msg, "||READY_TO_PLAY");
        send(socket, s_msg, BUFF_LEN, 0);
        if (GAME->isGameReady()){
            bzero(s_msg, BUFF_LEN);
            strcat(s_msg, "\n=====\\n");
            strcat(s_msg, "            ИГРА НАЧИНАЕТСЯ!");
            strcat(s_msg, "\n=====\\n");
            strcat(s_msg, "|common|");
            pthread_mutex_lock(mutex);
            send_to_all(SUBS, s_msg, BUFF_LEN);
            GAME->setStatus(START);
            CONTEXT->set_lobby_status(lobby_id, true);
            pthread_mutex_unlock(mutex);
        }
    }
    continue;
}
}

```



```

if(p_status == WAITING || p_status == READY_TO_PLAY){
    //send(socket, "EmptyMes", BUFF_LEN, 0);
    continue;
}

if(g_status == JOB_MAKE){
    if(p_status == EMPLOYER){
        if(strncmp(request, "sendhist", 9) != 0){
            strcat(s_msg, "        Вы - работодатель!\n");
            strcat(s_msg, " В Вашей компании открыты следующие вакансии:\n");
            pthread_mutex_lock(mutex);
            GAME->PassCards(GAME->get_profs(), GAME->EmployInfo()->getProfs(),
EMPLOYER_PROFS_NUM);
            pthread_mutex_unlock(mutex);
            vector<Card*>* emp_profs = GAME->EmployInfo()->getProfs();
            for(vector<Card*>::const_iterator pr = emp_profs->begin(); pr !=
emp_profs->end(); pr++){
                strcat(s_msg, "        ");
                strcat(s_msg, (*pr)->get_text().c_str());
                strcat(s_msg, "\n");
            }
            strcat(s_msg, "        Придумайте историю Вашей компании!");
            strcat(s_msg, "|givehist");
            strcat(s_msg, "|EMPLOYER");
            send(socket, s_msg, BUFF_LEN, 0);
            continue;
        }
        pthread_mutex_lock(mutex);
        GAME->EmployInfo()->setManual((string)output);
        pthread_mutex_unlock(mutex);
        strcat(s_msg, "\n    История:\n");
        strcat(s_msg, GAME->EmployInfo()->getManual().c_str());
        strcat(s_msg, "\n");
        strcat(s_msg, GAME->EmployInfo()->print_profs().c_str());
        strcat(s_msg, "|common|");
        pthread_mutex_lock(mutex);
        send_to_all_except(SUBS, socket, s_msg, BUFF_LEN);
        pthread_mutex_unlock(mutex);
        bzero(s_msg, BUFF_LEN);
        send(socket, "||WAITING", BUFF_LEN, 0);
        pthread_mutex_lock(mutex);
        GAME->set_player_status(id, WAITING);
        GAME->setStatus(P_PRE);
        pthread_mutex_unlock(mutex);
        continue;
    }
    continue;
}

if(g_status == P_PRE){
    if(p_status == ANSWERING){
        if(strncmp(request, "claim", 6) == 0){
            int vn = -1;
            if(strncmp(output, "1", 1) == 0){
                vn = 0;
            }else if(strncmp(output, "2", 1) == 0){
                vn = 1;
            }else if(strncmp(output, "3", 1) == 0){
                vn = 2;
            }
            sprintf(s_msg, "    Соискатель %s претендует на вакансию %s|common|",

```

```

        GAME->get_player_nick(id).c_str(), GAME->EmployInfo()->getProfs()-
>at(vn)->get_text().c_str());
        pthread_mutex_lock(mutex);
        send_to_all_except(SUBS, socket, s_msg, BUFF_LEN);
        GAME->EmployInfo()->add_claim(vn, id);
        pthread_mutex_unlock(mutex);
        send(socket, "|claim|ANSWERING", BUFF_LEN, 0);
        continue;
    }
    if(strncmp(request, "readytoanswer", 14) != 0){
        pthread_mutex_lock(mutex);
        GAME->PassCards(GAME->get_skills(), GAME->getPlayer(id)->getSkills(),
SKILL_NUM);

        GAME->GiveEmojiToPlayer(GAME->getPlayer(id));
        pthread_mutex_unlock(mutex);
        vector<Card*>* p_skills = GAME->getPlayer(id)->getSkills();
        Card* emo = GAME->getPlayer(id)->getEmoji();
        strcat(s_msg, " ЭМОЦИЯ: ");
        strcat(s_msg, emo->get_text().c_str());
        strcat(s_msg, "\n");
        strcat(s_msg, " НАВЫКИ:\n");
        for(vector<Card*>::const_iterator sk = p_skills->begin(); sk !=
p_skills->end(); sk++){
            strcat(s_msg, " - ");
            strcat(s_msg, (*sk)->get_text().c_str());
            strcat(s_msg, "\n");
        }
        strcat(s_msg, "|areanswer");
        strcat(s_msg, "|ANSWERING");
        send(socket, s_msg, BUFF_LEN, 0);
        bzero(s_msg, BUFF_LEN);
        sprintf(s_msg, "    Соискатель %s готовится отвечать...\n|common|",
GAME->get_player_nick(id).c_str());
        pthread_mutex_lock(mutex);
        send_to_all_except(SUBS, socket, s_msg, BUFF_LEN);
        pthread_mutex_unlock(mutex);
        continue;
    }
    pthread_mutex_lock(mutex);
    GAME->setStatus(P_MAKE);
    pthread_mutex_unlock(mutex);
    strcat(s_msg, "|giveanswer");
    strcat(s_msg, "|ANSWERING");
    send(socket, s_msg, BUFF_LEN, 0);
    bzero(s_msg, BUFF_LEN);
    sprintf(s_msg, "    Соискатель %s пишет резюме...\n|common|", GAME-
>get_player_nick(id).c_str());
    pthread_mutex_lock(mutex);
    send_to_all_except(SUBS, socket, s_msg, BUFF_LEN);
    pthread_mutex_unlock(mutex);
    continue;
}
continue;
}

if(g_status == P_MAKE){
    if (p_status == ANSWERING){
        if(strncmp(request, "sendanswer", 14) != 0){
            continue;
        }
        sprintf(s_msg, "    %s:", GAME->get_player_nick(id).c_str());
        strcat(s_msg, "    ");
        strcat(s_msg, output);
    }
}

```

```

        strcat(s_msg, "\n\n    Время для вопросов.|common|");
        pthread_mutex_lock(mutex);
        send_to_all_except(SUBS, socket, s_msg, BUFF_LEN);
        GAME->setStatus(QUESTIONS);
        vector<Player*>* tmq = GAME->get_players();
        for(vector<Player*>::iterator pl = tmq->begin(); pl != tmq->end(); pl++){
            if((*pl)->get_id() != GAME->get_answering_id()){
                (*pl)->setStatus(QUESTIONING);
            }
        }
        pthread_mutex_unlock(mutex);
        bzero(s_msg, BUFF_LEN);
        send(socket, "\n\n    Время для вопросов.|print|", BUFF_LEN, 0);
        sleep(1);
        continue;
    }
    sleep(1);
    continue;
}

if(g_status == QUESTIONS){
    if(GAME->get_player_status(id) == QUESTIONING){
        if(strncmp(request, "noquest", 8) == 0){
            pthread_mutex_lock(mutex);
            GAME->set_player_status(id, WAITING);
            if(GAME->no_questions()){
                GAME->set_player_status(GAME->get_answering_id(), WAITING);
                GAME->setStatus(P_OPEN);
            }
            pthread_mutex_unlock(mutex);
            send(socket, "||WAITING", BUFF_LEN, 0);
            sleep(1);
            continue;
        }

        strcat(s_msg, GAME->get_player_nick(id).c_str());
        if(strncmp(request, "quest", 6) == 0){
            pthread_mutex_lock(mutex);
            GAME->add_question((string)output);
            pthread_mutex_unlock(mutex);
            strcat(s_msg, ": |gquest|QUESTIONING");
        }else{
            strcat(s_msg, ": ||QUESTIONING");
        }
        send(socket, s_msg, BUFF_LEN, 0);
        sleep(1);
        continue;
    }

    if(strncmp(request, "aquest", 7) == 0){
        strcat(s_msg, " ");
        strcat(s_msg, GAME->get_player_nick(id).c_str());
        strcat(s_msg, ":");
        strcat(s_msg, output);
        strcat(s_msg, "\n\n|common|");
        pthread_mutex_lock(mutex);
        send_to_all_except(SUBS, socket, s_msg, BUFF_LEN);
        GAME->rem_question();
        pthread_mutex_unlock(mutex);
        if(GAME->no_questions()){
            pthread_mutex_lock(mutex);
            GAME->set_player_status(GAME->get_answering_id(), WAITING);
            GAME->setStatus(P_OPEN);
        }
    }
}

```

```

        pthread_mutex_unlock(mutex);
        sleep(1);
        continue;
    }
    send(socket, " ", 2, 0);
    sleep(1);
    continue;
}

if(!(GAME->get_questions()->empty())){
    string qu = *(GAME->get_questions()->begin());
    sprintf(s_msg, "    %s\n|common|", qu.c_str());
    pthread_mutex_lock(mutex);
    send_to_all_except(SUBS, socket, s_msg, BUFF_LEN);
    pthread_mutex_unlock(mutex);
    bzero(s_msg, BUFF_LEN);
    sprintf(s_msg, "    %s\n", qu.c_str());
    strcat(s_msg, "|quest|ANSWERING");
    send(socket, s_msg, BUFF_LEN, 0);
}else{
    strcat(s_msg, "|equest|ANSWERING");
    send(socket, s_msg, BUFF_LEN, 0);
}
sleep(1);
continue;
}

if(g_status == SCORES){
    if(GAME->get_player_status(id) == SCORING){
        int score = 0;
        if(strncmp(request, "score", 6) == 0){
            if(strncmp(output, "1", 1) == 0){
                score = 1;
            }else if(strncmp(output, "2", 1) == 0){
                score = 2;
            }else if(strncmp(output, "3", 1) == 0){
                score = 3;
            }else if(strncmp(output, "4", 1) == 0){
                score = 4;
            }else if(strncmp(output, "5", 1) == 0){
                score = 5;
            }
            sprintf(s_msg, "    %s поставил оценку.\n|print|", GAME-
>get_player_nick(id).c_str());
            pthread_mutex_lock(mutex);
            send_to_all_except(SUBS, socket, s_msg, BUFF_LEN);
            pthread_mutex_unlock(mutex);
            bzero(s_msg, BUFF_LEN);
            send(socket, "||WAITING", BUFF_LEN, 0);
            pthread_mutex_lock(mutex);
            GAME->add_scoreb(score);
            GAME->set_player_status(id, WAITING);
            pthread_mutex_unlock(mutex);
            if(GAME->score_over()){
                GAME->getPlayer(GAME->get_answering_id()->addScore(GAME-
>get_scoreb()));
                sprintf(s_msg, "\n    %s получил %d очков!|common|", GAME-
>get_player_nick(GAME->get_answering_id()).c_str()
, GAME->get_scoreb() );
                pthread_mutex_lock(mutex);
                GAME->set_answering_num(GAME->get_answering_num() + 1);
                if(GAME->get_answering_id() == GAME->getEmployerId()){

```

```

        GAME->setStatus(JOB_CHOICE);
        GAME->set_player_status(GAME->getEmployerId(), EMPLOYER);
    }else{
        GAME->setStatus(P_PRE);
    }
    send_to_all(SUBS, s_msg, BUFF_LEN);
    pthread_mutex_unlock(mutex);
}
sleep(1);
continue;
}
send(socket, "|score|SCORING", BUFF_LEN, 0);
}
sleep(1);
continue;
}

if(g_status == JOB_CHOICE){
    if(p_status == EMPLOYER){
        if(strncmp(request, "jchoice", 8) == 0){
            // формат распаковки выбора: "1:2,2:1,3:3"
            char* token = strtok(output, ",");
            while (token != NULL) {
                int vac_num, player_id;
                sscanf(token, "%d:%d", &vac_num, &player_id);
                pthread_mutex_lock(mutex);
                GAME->EmployInfo()->add_assignment(vac_num - 1, player_id);
                pthread_mutex_unlock(mutex);
                token = strtok(NULL, ",");
            }
            strcat(s_msg, "\n");
            strcat(s_msg, GAME->assign_professions().c_str());
            strcat(s_msg, "|common|WAITING");
            pthread_mutex_lock(mutex);
            GAME->set_player_status(id, WAITING);
            GAME->setEmployer(GAME->getEmployer() + 1);
            GAME->setStatus(START);
            send_to_all(SUBS, s_msg, BUFF_LEN);
            pthread_mutex_unlock(mutex);
            sleep(1);
            continue;
        }

        bzero(s_msg, BUFF_LEN);
        vector<Card*>* vacancies = GAME->EmployInfo()->getProfs();

        sprintf(s_msg, "    Работодатель %s выбирает сотрудников...\n",
            GAME->get_player_nick(GAME->getEmployerId()).c_str());

        char claims_out[BUFF_LEN];
        for (size_t i = 0; i < vacancies->size(); i++) {
            char buffer[256];
            sprintf(buffer, "    Вакансия %ld: %s\n", i+1, vacancies->at(i)-
>get_text().c_str());
            strcat(claims_out, buffer);

            vector<int*>* claimants = GAME->EmployInfo()-
>get_claims_for_vacancy(i);
            strcat(claims_out, "    Претенденты: ");
            if (claimants && !claimants->empty()) {
                for (size_t j = 0; j < claimants->size(); j++) {
                    char id_str[10];
                    sprintf(id_str, "%d", claimants->at(j));

```

```

        strcat(claims_out, id_str);
        strcat(claims_out, ") ");
        strcat(claims_out, GAME->get_player_nick(claimants-
>at(j)).c_str());
        if (j < claimants->size() - 1) strcat(claims_out, ", ");
    }
} else {
    strcat(claims_out, "нет");
}
strcat(claims_out, "\n\n");
}
strcat(s_msg, claims_out);
strcat(s_msg, "|common|");
pthread_mutex_lock(mutex);
send_to_all_except(SUBS, socket, s_msg, BUFF_LEN);
pthread_mutex_unlock(mutex);
bzero(s_msg, BUFF_LEN);
strcat(s_msg, " Введите Ваш выбор в формате: номер_вакансии:номер_игрока
(через запятую)\n");
strcat(s_msg, " Пример: 1:2,2:1,3:3\n");
strcat(s_msg, claims_out);
strcat(s_msg, "|givejchoice|EMPLOYER");
send(socket, s_msg, BUFF_LEN, 0);
}
sleep(1);
continue;
}
}
sleep(3);
close(socket);
pthread_exit(0);
}

void* lobby_thread(void* arg)
{
    int sm_socket = socket(AF_INET, SOCK_STREAM, 0);
    if (sm_socket < 0) {
        cout << " ОШИБКА: НЕ УДАЛОСЬ СОЗДАТЬ ЛОВБИ!" << endl;
        close(sm_socket);
        pthread_exit(0);
    }
    struct sockaddr_in s_addr;

    bzero((char*)&s_addr, sizeof(struct sockaddr_in));
    s_addr.sin_family = AF_INET;
    s_addr.sin_addr.s_addr = htonl(INADDR_ANY);
    s_addr.sin_port = 0;
    if(bind(sm_socket, (sockaddr*)&s_addr, sizeof(struct sockaddr_in)) < 0){
        cout << " ОШИБКА: НЕ УДАЛОСЬ ИНИЦИАЛИЗИРОВАТЬ ЛОВБИ!" << endl;
        close(sm_socket);
        pthread_exit(0);
    }
    unsigned int s_len = sizeof(struct sockaddr_in);
    if (getsockname(sm_socket, (struct sockaddr*)&s_addr, &s_len) < 0){
        cout << " ОШИБКА: НЕ УДАЛОСЬ НАЙТИ ПОРТ ЛОВБИ!" << endl;
        close(sm_socket);
        pthread_exit(0);
    }

    lobby_args* largs = (lobby_args*)arg;
    int lobby_id = largs->id;
    int lobby_size = largs->size;

```

```

StartupDbContext* CONTEXT = largs->context;
CONTEXT->set_lobby_port(lobby_id, ntohs(s_addr.sin_port));
pthread_mutex_t gmutex;
pthread_mutex_init(&gmutex, 0);

vector<int>* SUBS = new vector<int>;
Game* GAME = new Game(lobby_size);

srand(time(NULL));

int ss_socket = 0;

if(listen(sm_socket, lobby_size) < 0){
    cout << "    ОШИБКА: НЕ УДАЛОСЬ ОТКРЫТЬ ЛОВБИ!" << endl;
    pthread_mutex_destroy(&gmutex);
    close(sm_socket);
    pthread_exit(0);
}

GAME->setStatus(PRE);
int status;
char s_msg[BUFF_LEN];
for(;;)
{
    status = GAME->getStatus();
    if (status == PRE){
        ss_socket = accept(sm_socket, 0, 0);
        pthread_t thread_id;
        player_args pargs;
        pargs.game = GAME;
        pargs.subs = SUBS;
        pargs.socket = ss_socket;
        pargs.context = CONTEXT;
        pargs.lobby_id = lobby_id;
        pargs.mutex = &gmutex;
        pthread_create(&thread_id, NULL, player_thread, (void*)&pargs);
        pthread_detach(thread_id);
        sleep(1);
        continue;
    }
    if(status == START){
        sleep(1);
        if((size_t)GAME->getEmployer() >= GAME->get_players()->size()){
            pthread_mutex_lock(&gmutex);
            GAME->setStatus(OVER);
            pthread_mutex_unlock(&gmutex);
            continue;
        }
        pthread_mutex_lock(&gmutex);
        GAME->drop_cards();
        pthread_mutex_unlock(&gmutex);
        int emp = GAME->getEmployerId();
        sprintf(s_msg,
"%s=====\\n          Раунд
%d:\\n=====\\n          Работодатель: %s\\n
Работодатель придумывает историю своей компании...|common|\\n",
        GAME->print_players().c_str(), GAME->getEmployer() + 1, GAME-
>get_player_nick(emp).c_str());
        pthread_mutex_lock(&gmutex);
        GAME->set_player_status(emp, EMPLOYER);
        GAME->setStatus(JOB_MAKE);
        GAME->set_answering_num(1);
        send to all(SUBS, s_msg, BUFF_LEN);

```

```

        pthread_mutex_unlock(&gmutex);
        continue;
    }
    if (status == P_PRE){
        pthread_mutex_lock(&gmutex);
        GAME->set_player_status(GAME->get_answering_id(), ANSWERING);
        pthread_mutex_unlock(&gmutex);
        continue;
    }
    if(status == P_OPEN){
        sprintf(s_msg, "\n%s\n\n    Время для выставления оценок!\n|common|", GAME-
>open_p(GAME->get_answering_id()).c_str());
        pthread_mutex_lock(&gmutex);
        send_to_all(SUBS, s_msg, BUFF_LEN);
        GAME->set_scoreb(0);
        GAME->setStatus(SCORES);
        vector<Player*>* tms = GAME->get_players();
        for(vector<Player*>::iterator pl = tms->begin(); pl != tms->end(); pl++){
            if((*pl)->get_id() != GAME->get_answering_id()){
                (*pl)->setStatus(SCORING);
            }
        }
        pthread_mutex_unlock(&gmutex);
        continue;
    }
    if(status == OVER){
        bzero(s_msg, BUFF_LEN);
        strcat(s_msg, GAME->Endgame(CONTEXT).c_str());
        strcat(s_msg, "|over|");
        pthread_mutex_lock(&gmutex);
        send_to_all(SUBS, s_msg, BUFF_LEN);
        CONTEXT->rm_lobby(lobby_id);
        pthread_mutex_unlock(&gmutex);
        sleep(1);
        break;
    }
}
sleep(5);
pthread_mutex_destroy(&gmutex);
close(sm_socket);
pthread_exit(0);
}

void send_to_all(vector<int*>* s_sockets, char* msg, int mlen){
    for(vector<int*>::iterator it = s_sockets->begin(); it != s_sockets->end(); it++){
        send(*it, msg, mlen, 0);
    }
}

void send_to_all_exept(vector<int*>* s_sockets, int exep, char* msg, int mlen){
    for(vector<int*>::iterator it = s_sockets->begin(); it != s_sockets->end(); it++){
        if(*it != exep){
            send(*it, msg, mlen, 0);
        }
    }
}

// Структура сообщения (клиент -> сервер) "output|request"
void ser_decode_msg(char* msg, int mlen, char* output, char* request){
    bzero(output, mlen);
    bzero(request, mlen);
    int bc = get_line_b(output, msg, 0, mlen, '|');

```



```

    bc = get_line_b(request, msg, bc, mlen, ' ');
}

```

### 5.3. Библиотека “Game”

#### game.h

```

// СТАРТАП-ИГРА (БИБЛИОТЕКА)

#include <iostream>
#include <cstdio>
#include <string>
#include <vector>
#include <cstring>
#include <unistd.h>
#include <algorithm>
#include <bits/stdc++.h> // для рандомайзера
#include "dbcontext.h"

using namespace std;

#define BUFF_LEN 4096
#define OUT_LEN 4096-300
#define REQ_LEN 150
#define STA_LEN 150
#define MIN_P 3
#define MAX_P 3
#define EMPLOYER_PROFS_NUM 3
#define SKILL_NUM 4
#define REWARD_K 5

enum game_status {
    PRE = 0, // набор лобби.
    FULL,
    START, // все игроки выразили готовность начать игру.
    // ИГРОВОЙ ЦИКЛ:
    //-----
    JOB_MAKE, // работодатель придумывает историю предприятия.
    //Цикл выступления соискателей:
    //-----
    P_PRE, // Игрок готовится выступить.
    P_MAKE, // Игрок выступает.
    QUESTIONS, // Режим принятия вопросов. Если ещё есть вопросы, то QUESTIOS ->
P_ANSWER
    P_ANSWER, // Игрок отвечает на первый пришедший вопрос P_ANSWER -> QUESTIONS
    P_OPEN, // Карты игрока вскрываются.
    SCORES, // Выставление оценки.
    //-----
    JOB_CHOICE, // Работодатель выбирает
    //-----
    OVER,

```

```

    AFTERGAME
};

enum player_status {
    WAIT_ACCEPT = 0,
    PRE_TO_PLAY,
    READY_TO_PLAY,
    WAITING,
    EMPLOYER,
    ANSWERING,
    QUESTIONING,
    SCORING,
    LEFT
};

//-----
int get_line_b(char*, char*, int, int, char);
//-----

class Card
{
    friend ostream& operator<<(ostream& os, const Card& c);
public:
    Card(string _text_1);
    ~Card();
    string get_text() const;
private:
    string text;
};

class Employ_Info
{
public:
    Employ_Info();
    ~Employ_Info();
    vector<Card*>* getProfs() const;
    string print_profs() const;
    string getManual() const;
    void setManual(string n_man);
    void add_claim(int vac, int id);

    vector<int*>* get_claims_for_vacancy(int vac_index);
    void clear_claims();

    map<int, int> assignments;
    void add_assignment(int vac, int player);
    void clear_assignments();
    int get_assignment(int vac);
private:
    vector<Card*>* e_profs;
    string manual;
    vector<int*>* claim_matrix;
};

class Player
{
    friend ostream& operator<<(ostream& os, const Player& p);
public:
    Player(string _name, int _id);
    ~Player();

    void addScore(int _score);

```

```

void addSkill(Card* sk);
void addProf(Card* pr);
void addEmoji(Card* ej);

void remSkill(Card* sk);
void remEmoji();

int get_id() const;
string get_nick() const;

void setStatus(int ns);
int getStatus() const;
int getScore() const;

vector<Card*>* getSkills() const;
Card* getEmoji() const;

string print_skills() const;
string print_profs();

private:
    int id;
    string name;
    int score = 0;
    vector<Card*>* p_skills;
    vector<Card*>* p_profs;
    Card* p_emoji;
    int status = PRE_TO_PLAY;
};

class Game
{
    public:
        Game(int _pmax);
        ~Game();
        void game_init();
        vector<Card*>* get_profs() const;
        vector<Card*>* get_skills() const;
        vector<Card*>* get_emoji() const;
        vector<Player*>* get_players() const;
        void print_profs() const;
        void print_skills() const;
        void print_emoji() const;
        string print_players() const;
        int getStatus() const;
        int getPnum() const;
        int getRnum() const;
        int getMnum() const;
        int get_player_id(char* nick) const;
        string get_player_nick(int id) const;
        int get_player_status(int id) const;
        bool isGameReady() const;
        string get_players_list() const;

        void set_player_status(int id, int ns);
        void setStatus(int ns);
        string addPlayer(char* nick, int id);
        string remPlayer(int id);

        Player* getPlayer(int id) const;
        int getEmployerId() const;
        void setEmployer(int ne);
        int getEmployer() const;

```

```

Employ_Info* EmployInfo();
void ShuffleCards(vector<Card*>* cards);
void PassCards(vector<Card*>* giver, vector<Card*>* accepter, int n_cards);
void GiveEmojiToPlayer(Player* player);

void set_answering_num(int na);
int get_answering_num() const;
int get_answering_id() const;

vector<string*> get_questions() const;
void add_question(string q);
void rem_question();
bool no_questions() const;

string open_p(int id) const;

int get_scoreb() const;
void set_scoreb(int ns);
void add_scoreb(int as);
bool score_over() const;

string assign_professions();

void drop_cards();

string Endgame(StartupDbContext* context);

private:
    int p_max;
    int p_num;
    vector<Card*>* g_profs;
    vector<Card*>* g_skills;
    vector<Card*>* g_emoji;
    vector<Player*>* g_players;
    Employ_Info* g_employ;
    int status = PRE;
    int employer;
    int answering_num;
    vector<string*> g_questions;
    int score_buf;
};

```

#### game.cpp

```

// СТАРТАП-ИГРА (ИСХОДНЫЙ КОД)
#include "game.h"

int get_line_b(char* n_line, char* o_line, int start, int len, char b){
    if (start >= len) {
        n_line[0] = '\\0';
        return len;
    }

    int i = 0;
    int j = start;

```

```

while (j < len && i < len - 1) {
    char c = o_line[j];
    if (c == b) {
        strcat(n_line, "\\0");
        return j + 1;
    }
    n_line[i] = c;
    i++;
    j++;
}

//n_line[i] = '\\0';
strcat(n_line, "\\0");
return len - 1;
}

ostream& operator<<(ostream& os, const Card& c){
    os << c.text;
    return os;
}

Card::Card(string _text_1): text(_text_1){}
Card::~~Card(){};

string Card::get_text() const{
    return text;
}

ostream& operator<<(ostream& os, const Player& p){
    os << p.name << " (";
    for(vector<Card*>::iterator it = p.p_profs->begin(); it != p.p_profs->end(); it++){
        os << (**it) << ", ";
    }
    os << ")" << ": " << p.score;
    if (p.getStatus() == LEFT){
        os << " (Вышел) ";
    }
    return os;
}

Player::Player(string _name, int _id):
    id(_id),
    name(_name),
    score(0),
    p_skills(new vector<Card*>),
    p_profs(new vector<Card*>),
    p_emoji(nullptr),
    status(WAIT_ACCEPT)
{}

Player::~~Player(){
    p_skills->clear();
    delete p_skills;

    p_profs->clear();
    delete p_profs;

    delete p_emoji;
};

void Player::addScore(int _score){

```

```

        score += _score;
    }

    void Player::addSkill(Card* sk){
        p_skills->push_back(sk);
    }

    void Player::addProf(Card* pr){
        p_profs->push_back(pr);
    }

    void Player::addEmoji(Card* ej){
        p_emoji = ej;
    }

    void Player::remEmoji(){
        p_emoji = nullptr;
    }

    int Player::get_id() const{
        return id;
    }

    string Player::get_nick() const{
        return name;
    }

    void Player::remSkill(Card* sk){
        for(vector<Card*>::iterator c = p_skills->begin(); c != p_skills->begin(); c++){
            if (*c == sk){
                p_skills->erase(c);
                break;
            }
        }
    }

    void Player::setStatus(int ns){
        status = ns;
    }

    int Player::getStatus() const{
        return status;
    }

    int Player::getScore() const{
        return score;
    }

    string Player::print_skills() const{
        string res;
        if(p_skills){
            for(vector<Card*>::const_iterator sk = p_skills->begin(); sk != p_skills->end();
sk++){
                //cout << "      - " << **sk << endl;
                res += "      - ";
                res += (**sk).get_text();
                res += "\n";
            }
        }
        return res;
    }

    string Player::print_profs() {

```

```

string result;
if(p_profs && !p_profs->empty()){
    for(auto it = p_profs->begin(); it != p_profs->end(); ++it){
        if(it != p_profs->begin()){
            result += ", ";
        }
        result += (*it)->get_text();
    }
}
return result;
}

vector<Card*>* Player::getSkills() const{
    return p_skills;
}

Card* Player::getEmoji() const{
    return p_emoji;
}

Game::Game(int _pmax):
    p_max(_pmax),
    p_num(0),
    g_profs(nullptr),
    g_skills(nullptr),
    g_emoji(nullptr),
    g_players(nullptr),
    g_employ(nullptr),
    status(PRE),
    employer(0),
    answering_num(0),
    g_questions(nullptr),
    score_buf(0)
{
    game_init();
}

void Game::game_init()
{
    //-----
    g_profs = new vector<Card*>;
    g_skills = new vector<Card*>;
    g_emoji = new vector<Card*>;
    g_players = new vector<Player*>;
    g_employ = new Employ_Info;
    g_questions = new vector<string>;
    //-----

    g_profs->push_back(new Card("🐟 Рыбак"));
    g_profs->push_back(new Card("🗣️ Конферансье"));
    g_profs->push_back(new Card("👶 Малыш"));
    g_profs->push_back(new Card("✈️ Военный"));
    g_profs->push_back(new Card("📖 Библиотекарь"));
    g_profs->push_back(new Card("✈️ Пилот"));
    g_profs->push_back(new Card("🎪 Фокусник"));
    g_profs->push_back(new Card("👨 Фермер"));
    g_profs->push_back(new Card("🔧 Сантехник"));
    g_profs->push_back(new Card("📧 Почтальон"));
    g_profs->push_back(new Card("💇 Стилист"));
    g_profs->push_back(new Card("📖 Экскурсовод"));

```

```

g_profs->push_back(new Card("🚒 Гонщик"));
g_profs->push_back(new Card("🆘 Спасатель"));
g_profs->push_back(new Card("📁 Офисный менеджер"));
g_profs->push_back(new Card("🧪 Химик"));
g_profs->push_back(new Card("📷 Фотограф"));
g_profs->push_back(new Card("🎵 Музыкант"));
g_profs->push_back(new Card("👮 Полицейский"));
g_profs->push_back(new Card("🌿 Садовник"));
g_profs->push_back(new Card("⚖️ Адвокат"));
g_profs->push_back(new Card("📰 Журналист"));
g_profs->push_back(new Card("📦 Доставщик"));
g_profs->push_back(new Card("🎭 Актер"));
g_profs->push_back(new Card("🛒 Продавец"));
g_profs->push_back(new Card("🏆 Спортсмен"));
g_profs->push_back(new Card("🔒 Охранник"));
g_profs->push_back(new Card("👗 Модельер"));
g_profs->push_back(new Card("👷 Дворник"));
g_profs->push_back(new Card("👨‍🔥 Пожарный"));
g_profs->push_back(new Card("📊 Бухгалтер"));
g_profs->push_back(new Card("🎙️ Радиоведущий"));
g_profs->push_back(new Card("☕ Бариста"));
g_profs->push_back(new Card("🐔 Пастух"));
g_profs->push_back(new Card("🎨 Художник"));
g_profs->push_back(new Card("👨‍🍳 Повар"));
g_profs->push_back(new Card("🏗️ Строитель"));
g_profs->push_back(new Card("📖 Учитель"));
g_profs->push_back(new Card("🤡 Клоун"));
g_profs->push_back(new Card("🏠 Домработница"));
g_profs->push_back(new Card("⚓ Моряк"));
g_profs->push_back(new Card("📝 Поэт"));
g_profs->push_back(new Card("👨‍⚕️ Доктор"));
g_profs->push_back(new Card("🚀 Космонавт"));
g_profs->push_back(new Card("🔨 Кузнец"));
g_profs->push_back(new Card("💻 Программист"));
g_profs->push_back(new Card("🔧 Автомеханик"));
g_profs->push_back(new Card("⚙️ Инженер"));
g_profs->push_back(new Card("👤 Официант"));
g_profs->push_back(new Card("👩‍💃 Медсестра"));
ShuffleCards(g_profs);
//-----
g_skills->push_back(new Card("У меня острое зрение."));
g_skills->push_back(new Card("Я быстро печатаю на клавиатуре."));
g_skills->push_back(new Card("Я умею держать себя в руках."));
g_skills->push_back(new Card("Я хорошо ориентируюсь на местности."));
g_skills->push_back(new Card("Я хорошо разбираюсь в налогах."));
g_skills->push_back(new Card("Я могу запоминать большие объёмы информации."));
g_skills->push_back(new Card("Я умею аргументированно спорить."));
g_skills->push_back(new Card("Я умею пропалывать грядки."));
g_skills->push_back(new Card("Я знаю столицы всех стран."));
g_skills->push_back(new Card("У меня заразительный смех."));
g_skills->push_back(new Card("Я с душой рассказываю стихи."));
g_skills->push_back(new Card("Я могу найти всё, что угодно в интернете."));
g_skills->push_back(new Card("Я быстро бегаю."));
g_skills->push_back(new Card("Я умею ухаживать за цветами."));
g_skills->push_back(new Card("Я умею вести переговоры."));

```



```

g_skills->push_back(new Card("Я умею художественно свистеть."));
g_skills->push_back(new Card("Я умею чинить всё, что угодно."));
g_skills->push_back(new Card("Я могу прыгать через скакалку целый день."));
g_skills->push_back(new Card("Я умею находить общий язык с детьми."));
g_skills->push_back(new Card("Я никогда не опаздываю."));
g_skills->push_back(new Card("Я хорошо разбираюсь в технике."));
g_skills->push_back(new Card("Я умею танцевать вальс."));
g_skills->push_back(new Card("Я безошибочно считаю в уме."));
g_skills->push_back(new Card("Я умею рисовать."));
g_skills->push_back(new Card("У меня хорошее чувство ритма."));
g_skills->push_back(new Card("У меня каллиграфический почерк."));
g_skills->push_back(new Card("Я умею организовывать своё время."));
g_skills->push_back(new Card("Я умею садиться на шпагат."));
g_skills->push_back(new Card("Я очень доходчиво объясняю."));
g_skills->push_back(new Card("Я умею внимательно слушать."));
g_skills->push_back(new Card("Я умею вязать морские узлы."));
g_skills->push_back(new Card("Я хорошо фотографирую."));
g_skills->push_back(new Card("Я знаю расписание автобусов наизусть."));
g_skills->push_back(new Card("Я прирождённый оратор."));
g_skills->push_back(new Card("Я свободно говорю на пяти языках."));
g_skills->push_back(new Card("Я умею плавать."));
g_skills->push_back(new Card("Я умею вкусно готовить."));
g_skills->push_back(new Card("Я могу стоять на одной ноге с закрытыми глазами."));
g_skills->push_back(new Card("У меня аналитический склад ума."));
g_skills->push_back(new Card("Я легко нахожу общий язык с людьми."));
g_skills->push_back(new Card("Я умею определять птиц по голосам."));
g_skills->push_back(new Card("Я умею рубить дрова."));
g_skills->push_back(new Card("Я умею управлять повозкой с лошадью."));
g_skills->push_back(new Card("Я умею лепить из пластилина."));
g_skills->push_back(new Card("Я красиво пою."));
g_skills->push_back(new Card("Я легко завожу новые знакомства."));
g_skills->push_back(new Card("Я творчески подхожу к любой задаче."));
g_skills->push_back(new Card("Я умею играть в шахматы."));
g_skills->push_back(new Card("Я умею вышивать крестиком."));
g_skills->push_back(new Card("Я умею ездить на мотоцикле."));
g_skills->push_back(new Card("Я хорошо знаю физику."));
g_skills->push_back(new Card("Я умею кататься на коньках."));
g_skills->push_back(new Card("Я умею лепить куличики."));
g_skills->push_back(new Card("Я умею отличать правду от лжи."));
g_skills->push_back(new Card("Я умею хорошо знать химию."));
g_skills->push_back(new Card("Я легко перевоплощаюсь."));
g_skills->push_back(new Card("Я умею предсказывать погоду."));
g_skills->push_back(new Card("Я виртуозно играю на скрипке."));
g_skills->push_back(new Card("Я умею делать массаж."));
g_skills->push_back(new Card("Я умею оказывать первую помощь."));
g_skills->push_back(new Card("Я умею надувать большие пузыри из жвачки."));
g_skills->push_back(new Card("Я хорошо знаю законы."));
g_skills->push_back(new Card("Я умею делать причёски."));
g_skills->push_back(new Card("Я умею выявлять преимущества положения."));
g_skills->push_back(new Card("Я умею показывать фокусы."));
g_skills->push_back(new Card("Я умею метко стрелять."));
g_skills->push_back(new Card("Я умею разжигать костёр."));
g_skills->push_back(new Card("Я умею отличать съедобные грибы от ядовитых."));
g_skills->push_back(new Card("Я притягиваю удачу."));
g_skills->push_back(new Card("Я умею убеждать."));
ShuffleCards(g_skills);
//-----
g_emoji->push_back(new Card("Вы во всём сомневаетесь, даже в собственных
способностях."));
g_emoji->push_back(new Card("Вам всё лень."));
g_emoji->push_back(new Card("Вы чувствуете себя очень виноватым."));
g_emoji->push_back(new Card("У вас много слов-паразитов: ну, как бы, это, типа, в
общем..."));

```

```

g_emoji->push_back(new Card("Вы изо всех сил стараетесь понравиться."));
g_emoji->push_back(new Card("Вокруг вас летает назойливая муха."));
g_emoji->push_back(new Card("Вы хотите спать и никак не можете взбодриться."));
g_emoji->push_back(new Card("Вы много смеётесь по поводу и без него."));
g_emoji->push_back(new Card("Вы зануда."));
g_emoji->push_back(new Card("Вы говорите с акцентом."));
g_emoji->push_back(new Card("Вы сильно торопитесь."));
g_emoji->push_back(new Card("Вам холодно."));
g_emoji->push_back(new Card("Вы рассеяны, постоянно что-то забываете, путаете и
теряете."));
g_emoji->push_back(new Card("Вы постоянно переспрашиваете."));
g_emoji->push_back(new Card("Вы ведёте себя развязно, дерзко."));
g_emoji->push_back(new Card("Вы часто используете уменьшительно-ласкательные
формы: счётик, грабельки, микрофончик..."));
g_emoji->push_back(new Card("Вы гордитесь своими детьми и хвалите их
успехами"));
g_emoji->push_back(new Card("У вас меланхолия, ничто не радует, нет в жизни
счастья."));
g_emoji->push_back(new Card("Вы ёрзаете на стуле."));
g_emoji->push_back(new Card("У вас криминальное прошлое."));
g_emoji->push_back(new Card("Вы слишком самоуверенны, всем вокруг с вами
невероятно повезло."));
g_emoji->push_back(new Card("Вам жарко"));
g_emoji->push_back(new Card("Вы боитесь общественного мнения."));
g_emoji->push_back(new Card("Вы сентиментальны, легко можете растрогаться и даже
всплакнуть."));
g_emoji->push_back(new Card("Вы говорите слишком тихо."));
g_emoji->push_back(new Card("Вы жуёте жвачку."));
g_emoji->push_back(new Card("Вы чрезвычайно нервничаете."));
g_emoji->push_back(new Card("Вы постоянно поправляете причёску."));
g_emoji->push_back(new Card("Вы голодны."));
g_emoji->push_back(new Card("Вы говорите по-простецки, не церемонясь."));
g_emoji->push_back(new Card("Вы чешетесь."));
g_emoji->push_back(new Card("Вы обо всех заботитесь, всем сочувствуете и
сопереживаете."));
g_emoji->push_back(new Card("У вас апатия, всё безразлично."));
g_emoji->push_back(new Card("У вас приступ панического страха."));
g_emoji->push_back(new Card("Вам трудно угодить, вы избирательны и капризны."));
g_emoji->push_back(new Card("Вы слишком громко говорите."));
g_emoji->push_back(new Card("Вы сильно простужены."));
g_emoji->push_back(new Card("Вы склонны всё преувеличивать."));
g_emoji->push_back(new Card("Вы вот-вот чихнёте."));
g_emoji->push_back(new Card("Вас всё вокруг раздражает."));
g_emoji->push_back(new Card("Вы излагаете мысли быстро, активно, нетерпеливо."));
g_emoji->push_back(new Card("Вы говорите самозабвенно, не обращая ни на что и ни
на кого внимания."));
g_emoji->push_back(new Card("Вы икаете."));
g_emoji->push_back(new Card("У вас дислексия."));
g_emoji->push_back(new Card("Вы любите сокращения."));
g_emoji->push_back(new Card("Вы постоянно напевааете."));
g_emoji->push_back(new Card("Вы плохо говорите по-русски."));
g_emoji->push_back(new Card("Вы льете воду."));
g_emoji->push_back(new Card("Вы презираете других соискателей."));
g_emoji->push_back(new Card("Вы хотите много денег."));
ShuffleCards(g_emoji);

status = PRE;
employer = 0;
}

Game::~Game()
{
    if(g_profs){

```

```

        g_profs->clear();
        delete g_profs;
    }
    if(g_skills){
        g_skills->clear();
        delete g_skills;
    }
    if(g_emoji){
        g_emoji->clear();
        delete g_emoji;
    }
    if(g_players){
        g_players->clear();
        delete g_players;
    }
}

vector<Card*>* Game::get_profs() const{
    return g_profs;
}

vector<Card*>* Game::get_skills() const{
    return g_skills;
}

vector<Card*>* Game::get_emoji() const{
    return g_emoji;
}

vector<Player*>* Game::get_players() const{
    return g_players;
}

void Game::print_profs() const{
    if(g_profs){
        for(vector<Card*>::const_iterator pr = g_profs->begin(); pr != g_profs->end();
pr++){
            cout << **pr << endl;
        }
    }
}

void Game::print_skills() const{
    if(g_skills){
        for(vector<Card*>::const_iterator sk = g_skills->begin(); sk != g_skills->end();
sk++){
            cout << *sk << endl;
        }
    }
}

void Game::print_emoji() const{
    if(g_emoji){
        for(vector<Card*>::const_iterator ej = g_emoji->begin(); ej != g_emoji->end();
ej++){
            cout << *ej << endl;
        }
    }
}

string Game::print_players() const {
    string result;

    if (g_players == nullptr || g_players->empty()){
        result = "    НЕТ ИГРОКОВ";
    }
}

```

```

        return result;
    }

    result += "=====ИГРОКИ=====\n";

    for(auto p = g_players->begin(); p != g_players->end(); ++p){
        char buffer[BUFF_LEN];
        snprintf(buffer, BUFF_LEN,
            "    %s (%s):    %d    %s\n",
            (*p)->get_nick().c_str(),
            (*p)->print_profs().c_str(),
            (*p)->getScore(),
            (get_player_status((*p)->get_id()) == LEFT) ? "(Вышел)" : "");
        result += buffer;
    }

    result += "=====\n";
    return result;
}

int Game::getStatus() const{
    return status;
}

void Game::setStatus(int ns){
    status = ns;
}

string Game::addPlayer(char* nick, int id){
    Player* p = new Player(nick, id);
    g_players->push_back(p);
    p_num = getPnum();

    char buffer[BUFF_LEN];
    snprintf(buffer, BUFF_LEN,
        "    %s присоединился к игре!\n    Игроков: %d / %d\n    Готовы: %d / %d\n\n",
        nick, getPnum(), p_max, getRnum(), p_max);

    if(getPnum() >= MAX_P){
        setStatus(FULL);
    }

    return string(buffer);
}

int Game::getPnum() const{
    return g_players->size();
}

int Game::getMnum() const{
    return p_max;
}

int Game::get_player_id(char* nick) const{
    for(vector<Player*>::iterator p = g_players->begin(); p != g_players->end(); p++){
        if(strcmp((*p)->get_nick().c_str(), nick) == 0){
            return (*p)->get_id();
        }
    }
    return -1;
}

string Game::get_player_nick(int id) const{

```

```

    for(vector<Player*>::iterator p = g_players->begin(); p != g_players->end(); p++){
        if((*p)->get_id() == id){
            return (*p)->get_nick();
        }
    }
    return NULL;
}

int Game::get_player_status(int id) const{
    for(vector<Player*>::iterator p = g_players->begin(); p != g_players->end(); p++){
        if((*p)->get_id() == id){
            return (*p)->getStatus();
        }
    }
    return WAIT_ACCEPT;
}

string Game::remPlayer(int id){
    string result;

    for(auto it = g_players->begin(); it != g_players->end(); ++it){
        if((*it)->get_id() == id){
            char buffer[BUFF_LEN];
            sprintf(buffer, "    %s покинул игру.\n", get_player_nick(id).c_str());
            result += buffer;

            if (getStatus() == PRE || getStatus() == FULL){
                delete *it;
                g_players->erase(it);

                char buf[BUFF_LEN];
                sprintf(buf, "    Игроков: %d / %d\n    Готовы: %d / %d\n",
                    getPnum(), p_max, getRnum(), p_max);
                result += buf;
            } else {
                (*it)->setStatus(LEFT);
                setStatus(OVER);
            }
            return result;
        }
    }

    return "    Игрок не найден.\n";
}

void Game::set_player_status(int id, int ns){
    for(vector<Player*>::iterator p = g_players->begin(); p != g_players->end(); p++){
        if((*p)->get_id() == id){
            (*p)->setStatus(ns);
        }
    }
}

int Game::getRnum() const{
    int ready = 0;
    for(vector<Player*>::const_iterator it = g_players->begin(); it != g_players->end();
it++){
        if ((*it)->getStatus() == READY_TO_PLAY){
            ready++;
        }
    }
    return ready;
}

```

```

bool Game::isGameReady() const{
    return ((getPnum() >= MIN_P ) && (getRnum() == getPnum()));
}

Player* Game::getPlayer(int id) const{
    for(vector<Player*>::iterator p = g_players->begin(); p != g_players->end(); p++){
        if((*p)->get_id() == id){
            return *p;
        }
    }
    return nullptr;
}

int Game::getEmployerId() const{
    return g_players->at(getEmployer())->get_id();
}

int Game::getEmployer() const{
    return employer;
}

void Game::setEmployer(int ne){
    employer = ne;
}

void Game::PassCards(vector<Card*>* giver, vector<Card*>* acceptor, int n_cards){
    Card* card;
    for(int i = 0; i < n_cards; i++){
        card = *(giver->begin());
        giver->erase(giver->begin());
        acceptor->push_back(card);
    }
}

void Game::GiveEmojiToPlayer(Player* player){
    if (g_emoji->empty()) {
        cout << "Ошибка: нет эмоций в колоде!" << endl;
        return;
    }
    Card* emo = g_emoji->front();
    g_emoji->erase(g_emoji->begin());
    player->addEmoji(emo);
}

void Game::ShuffleCards(vector<Card*>* cards){
    random_device rd;
    mt19937 g(rd());
    shuffle(cards->begin(), cards->end(), g);
}

Employ_Info* Game::EmployInfo(){
    return g_employ;
}

void Game::set_answering_num(int na){
    answering_num = na;
}

int Game::get_answering_num() const{
    return answering_num;
}

```

```

}

int Game::get_answering_id() const{
    if(getEmployer() + answering_num >= (int)g_players->size()){
        return g_players->at(getEmployer() + answering_num - g_players->size())->get_id();
    }
    return g_players->at(getEmployer() + answering_num)->get_id();
}

vector<string>* Game::get_questions() const{
    return g_questions;
}

void Game::add_question(string q){
    g_questions->push_back(q);
}

void Game::rem_question(){
    get_questions()->erase(g_questions->begin());
}

bool Game::no_questions() const{
    if(status != QUESTIONS){
        return true;
    }
    for(vector<Player*>::iterator p = g_players->begin(); p != g_players->end(); p++){
        if((*p)->getStatus() == QUESTIONING){
            return false;
        }
    }
    if(!(get_questions()->empty())){
        return false;
    }
    return true;
}

string Game::open_p(int id) const {
    string result;
    Player* p = getPlayer(id);
    result += "    Карты " + get_player_nick(id) + ":\n";
    if(p->getEmoji()){
        result += "    Эмоция: " + p->getEmoji()->get_text() + "\n";
    } else {
        result += "    Эмоция: (не получена)\n";
    }
    result += "    Навыки:\n";
    result += p->print_skills();
    return result;
}

int Game::get_scoreb() const{
    return score_buf;
}

void Game::set_scoreb(int ns){
    score_buf = ns;
}

void Game::add_scoreb(int as){
    score_buf += as;
}

bool Game::score_over() const{

```

```

    if(status != SCORES){
        return true;
    }
    for(vector<Player*>::iterator p = g_players->begin(); p != g_players->end(); p++){
        if((*p)->getStatus() == SCORING){
            return false;
        }
    }
    return true;
}

string Game::assign_professions() {
    string result;
    vector<Card*>* vacancies = g_employ->getProfs();

    for (size_t i = 0; i < vacancies->size(); i++) {
        int chosen_player_id = g_employ->get_assignment(i);
        vector<int>* claimants = g_employ->get_claims_for_vacancy(i);
        Card* profession = vacancies->at(i);

        if (!claimants || claimants->empty()) {
            char buffer[256];
            sprintf(buffer, "Вакансия \"%s\" никому не досталась (нет претендентов)\n",
                    profession->get_text().c_str());
            result += buffer;
            continue;
        }

        if (chosen_player_id != -1) {
            auto it = find(claimants->begin(), claimants->end(), chosen_player_id);
            if (it != claimants->end()) {
                Player* chosen_player = getPlayer(chosen_player_id);
                if (chosen_player) {
                    chosen_player->addProf(profession);
                    chosen_player->addScore(3);
                    char buffer[256];
                    sprintf(buffer, "Вакансия \"%s\" достаётся %s (выбор
работодателя)!\n",
                            profession->get_text().c_str(),
                            chosen_player->get_nick().c_str());
                    result += buffer;
                    continue;
                }
            } else {
                char buffer[256];
                sprintf(buffer, "Работодатель выбрал %s, но он не претендовал на вакансию
\"%s\"!\n",
                        getPlayer(chosen_player_id)->get_nick().c_str(),
                        profession->get_text().c_str());
                result += buffer;
            }
        }

        if (claimants && !claimants->empty()) {
            int random_index = rand() % claimants->size();
            int random_player_id = (*claimants)[random_index];
            Player* chosen_player = getPlayer(random_player_id);

            if (chosen_player) {
                chosen_player->addProf(profession);
                chosen_player->addScore(3);
                char buffer[256];

```



```

        sprintf(buffer, "Вакансия \"%s\" достаётся %s (случайный выбор среди
претендентов)!\n",
                profession->get_text().c_str(),
                chosen_player->get_nick().c_str());
        result += buffer;
    }
}

g_employ->clear_claims();
g_employ->clear_assignments();
vacancies->clear();

return result;
}

string Game::get_players_list() const {
    string result;
    for (auto p : *g_players) {
        if (p->getStatus() != LEFT && p->get_id() != getEmployerId()) {
            char buffer[256];
            sprintf(buffer, "%d) %s\n", p->get_id(), p->get_nick().c_str());
            result += buffer;
        }
    }
    return result;
}

void Game::drop_cards() {

    vector<Card*>* to_hand_s = new vector<Card*>;
    vector<Card*>* to_hand_e = new vector<Card*>;
    for (vector<Player*>::iterator pl = g_players->begin(); pl != g_players->end(); pl++) {
        if (!(*pl)->getSkills()->empty()) {
            PassCards((*pl)->getSkills(), to_hand_s, SKILL_NUM);
        }
        if ((*pl)->getEmoji() != nullptr) {
            to_hand_e->push_back((*pl)->getEmoji());
            (*pl)->remEmoji();
        }
    }
    if (!to_hand_s->empty()) {
        ShuffleCards(to_hand_s);
        PassCards(to_hand_s, g_skills, to_hand_s->size());
        delete to_hand_s;
    }
    if (!to_hand_e->empty()) {
        ShuffleCards(to_hand_e);
        PassCards(to_hand_e, g_emoji, to_hand_e->size());
        delete to_hand_e;
    }
}

string Game::Endgame(StartupDbContext* context) {
    string result;
    int max = 0;

    result += "=====\n";
    result += "            ИГРА ОКОНЧЕНА!\n";
    result += "=====\n";

    for (auto pl : *g_players) {

```

```

        if(pl->getStatus() != LEFT && pl->getScore() > max){
            max = pl->getScore();
        }
    }

    int winnum = 0;
    for(auto pl : *g_players){
        if(pl->getStatus() != LEFT && pl->getScore() == max){
            winnum++;
        }
    }

    if(winnum == 0){
        winnum = 1;
    }

    result += "\n";

    for(auto pl : *g_players){
        char buffer[256];
        int reward;

        if(pl->getStatus() == LEFT){
            reward = -REWARD_K * getPnum();
            sprintf(buffer, "%s (вышел из игры) - изменение рейтинга: %d\n",
                    pl->get_nick().c_str(), reward);
        }
        else if(pl->getScore() == max){
            reward = (REWARD_K * getPnum()) / winnum;
            sprintf(buffer, "%s - ПОБЕДИТЕЛЬ! Очки: %d, изменение рейтинга: +%d\n",
                    pl->get_nick().c_str(), pl->getScore(), reward);
        }
        else {
            reward = -REWARD_K / (getPnum() - winnum);
            sprintf(buffer, "%s - очки: %d, изменение рейтинга: %d\n",
                    pl->get_nick().c_str(), pl->getScore(), reward);
        }

        result += buffer;
        context->add_player_score(pl->get_nick(), reward);
    }

    result += "=====\n";

    return result;
}

Employ_Info::Employ_Info() {
    e_profs = new vector<Card*>;
    claim_matrix = new vector<int>[EMPLOYER_PROFS_NUM];
}

Employ_Info::~Employ_Info() {
    e_profs->clear();
    delete e_profs;
}

vector<Card*>* Employ_Info::getProfs() const{
    return e_profs;
}

```

```

string Employ_Info::getManual() const{
    return manual;
}

void Employ_Info::setManual(string n_man){
    manual = n_man;
}

string Employ_Info::print_profs() const{
    string res = "";
    if(e_profs){
        res += "-----\n";
        res += "Вакансии:\n";
        res += "-----\n";
        int i = 1;
        for(vector<Card*>::const_iterator pr = e_profs->begin(); pr != e_profs->end();
pr++){
            char buf[256];
            sprintf(buf, " %d) %s\n", i, (*pr)->get_text().c_str());
            res += buf;
            i++;
        }
        res += "-----\n";
    }
    return res;
}

void Employ_Info::add_claim(int vac, int id){
    claim_matrix[vac].push_back(id);
}

vector<int*> Employ_Info::get_claims_for_vacancy(int vac_index) {
    if (vac_index >= 0 && vac_index < EMPLOYER_PROFS_NUM) {
        return &claim_matrix[vac_index];
    }
    return nullptr;
}

void Employ_Info::clear_claims() {
    for (int i = 0; i < EMPLOYER_PROFS_NUM; i++) {
        claim_matrix[i].clear();
    }
}

void Employ_Info::add_assignment(int vac, int player){
    assignments[vac] = player;
}

void Employ_Info::clear_assignments(){
    assignments.clear();
}

int Employ_Info::get_assignment(int vac){
    return assignments.count(vac) ? assignments[vac] : -1;
}

```

## 5.4. Библиотека “Chat”

### chat.h

```
#include <iostream>
#include <cstdio>
#include <string>
#include <vector>
#include <cstring>
#include <unistd.h>
#include <algorithm>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <time.h>
#include <arpa/inet.h>
#include <pthread.h>

#define BUFF_LEN 4096
#define CHATS_DIR "info/chats/"

using namespace std;

typedef struct chat_args{
    int socket;
    string login;
}chat_args;

void* msg_thread(void* args);
void* msg_accept_thread(void* args);
```

### chat.cpp

```

#include "chat.h"

void* msg_thread(void* args){
    chat_args* margs = (chat_args*)args;
    int socket = margs->socket;
    string login = margs->login;
    char amsg[BUFF_LEN];
    char nick[BUFF_LEN];
    char smsg[BUFF_LEN];
    if(recv(socket, amsg, BUFF_LEN, 0) > 0){
        send(socket, "|received", BUFF_LEN, 0);
        sscanf(amsg, "%[^:]:%[^|]", nick, smsg);
        string chatname = CHATS_DIR;
        chatname += login;
        chatname += "/";
        chatname += nick;
        chatname += ".txt";
        FILE* f_chat = fopen(chatname.c_str(), "a");
        fprintf(f_chat, smsg);
        fclose(f_chat);
    }
    delete margs;
    close(socket);
    pthread_exit(0);
}

void* msg_accept_thread(void* args){
    chat_args* cargs = (chat_args*)args;
    int socket = cargs->socket;
    string login = cargs->login;

    for(;;)
    {
        int ssock = accept(socket, 0, 0);
        pthread_t mt;

        chat_args* margs = new chat_args();
        margs->socket = ssock;
        margs->login = login;

        pthread_create(&mt, NULL, msg_thread, (void*)margs);
        pthread_detach(mt);
    }
    close(socket);
    pthread_exit(0);
}

```

## 5.5. Библиотека “DbContext”

```

// КОНТЕКСТ БАЗЫ ДАННЫХ (БИБЛИОТЕКА)
#include <iostream>
#include <pqxx/pqxx>
#include <string>
#include <memory>
#include <sodium.h>

#define S_ADDRESS "localhost"
#define S_PORT 5432
#define DATABASE "StartupDatabase"

using namespace std;
using namespace pqxx;

enum login_status{
    L_WRONG = 0,
    L_ONLINE,
    L_SUCCESS
};

enum reg_status{
    R_BUSY = 0,
    R_SUCCESS,
    R_FAIL
};

typedef struct User {
    int ID;
    int socket;
    char* login;
    char* password;
    int online;
    int score;
} User;

typedef struct Lobby {
    int ID;
    int socket;
    char* name;
    int size;
} Lobby;

class StartupDbContext {
public:
    StartupDbContext(string _address, int _port, string _database, string _user,
string _password);
    ~StartupDbContext();

    connection* getConnection();
    bool isConnected();

    int auth(string login, string password, string addr);
    int reg(string login, string password);
    int logout(string login);
    string get_lobbies();
    string get_players_on();
    string get_players_all();
    string get_rating();

```

```

bool finduser(string nick, string& ip, int& port, bool& online);
bool setupport(string login, int port);

void set_lobby_num(int id, int nv);
int add_lobby(string creator, string name, int num);
int join_lobby(int id);
void rm_lobby(int id);
void set_lobby_port(int id, int port);
void set_lobby_status(int id, bool ready);
void add_player_score(string login, int score);

void clear_online();
void clear_lobbies();

private:
    string address;
    int port;
    connection* conn;
    string database;
    string user;
    string password;
    pthread_mutex_t db_mutex;
};

```

#### dbcontext.cpp

```

// КОНТЕКСТ БАЗЫ ДАННЫХ (ИСХОДНЫЙ КОД)
#include "dbcontext.h"

```

```

StartupDbContext::StartupDbContext(string _address, int _port, string _database, string _user,
string _password):
address(_address), port(_port), database(_database), user(_user), password(_password)
{
    string conn_str = "host=" + address + " " +
        "port=" + to_string(port) + " " +
        "dbname=" + database + " " +
        "user=" + user + " " +
        "password=" + password + " ";

    conn = new connection(conn_str);
    pthread_mutex_init(&db_mutex, 0);
    if(!conn->is_open()){
        cout << "ОШИБКА: НЕ УДАЛОСЬ УСТАНОВИТЬ СОЕДИНЕНИЕ" << endl;
        exit(1);
    }
    string init_script = "CREATE TABLE IF NOT EXISTS USERS (ID Serial Primary Key, " +
        (string)"login varchar(256), " +
        "password varchar(256), " +
        "online int, " +
        "score int, " +
        "address varchar(255), " +
        "port int); " +
        "CREATE TABLE IF NOT EXISTS GAMES (" +
        "ID Serial Primary Key, " +
        "name varchar(256), " +
        "size int, " + "busy int, port int, began bool, creator int);";

    work w(*conn);
    result res = w.exec(init_script);
    w.commit();
}

```

```

    cout << "СОЕДИНЕНИЕ С БАЗОЙ ДАННЫХ УСПЕШНО УСТАНОВЛЕНО!" << endl;
}

StartupDbContext::~StartupDbContext() {
    if(conn) {
        conn->close();
        delete conn;
        conn = nullptr;
        pthread_mutex_destroy(&db_mutex);
    }
}

connection* StartupDbContext::getConnection() {
    return conn;
}

bool StartupDbContext::isConnected() {
    return (conn != nullptr);
}

int StartupDbContext::auth(string login, string password, string addr) {
    pthread_mutex_lock(&db_mutex);
    if(!isConnected()) {
        cout << "ОШИБКА: НЕТ СОЕДИНЕНИЯ С БАЗОЙ!" << endl;
        pthread_mutex_unlock(&db_mutex);
        return -1;
    }

    string q = "SELECT * FROM users WHERE login = $1";
    work w(*conn);
    result res = w.exec_params(q, login);
    w.commit();

    if(res.empty()) {
        pthread_mutex_unlock(&db_mutex);
        return L_WRONG;
    }

    string stored_hash = res[0]["password"].as<string>();
    if (crypto_pwhash_str_verify(stored_hash.c_str(), password.c_str(), password.length()) != 0)
        pthread_mutex_unlock(&db_mutex);
    return L_WRONG;
}

if(res[0]["online"].as<int>() == 1) {
    pthread_mutex_unlock(&db_mutex);
    return L_ONLINE;
}

string update_q = "UPDATE users SET online = $1, address = $2 WHERE login = $3";
w.exec_params(update_q, 1, addr, login);
w.commit();
pthread_mutex_unlock(&db_mutex);
return L_SUCCESS;
}

int StartupDbContext::reg(string login, string password) {
    pthread_mutex_lock(&db_mutex);
    if(!isConnected()) {
        cout << "ОШИБКА: НЕТ СОЕДИНЕНИЯ С БАЗОЙ!" << endl;
        pthread_mutex_unlock(&db_mutex);
        return -1;
    }
}

```



```

}
string q = "SELECT * FROM users WHERE login = $1";
work w(*conn);
result res = w.exec_params(q, login);
w.commit();

if(!res.empty()){
    pthread_mutex_unlock(&db_mutex);
    return R_BUSY;
}

q = "INSERT INTO users (login, password, online, score) VALUES ($1, $2, $3, $4)";
res = w.exec_params(q, login, password, 0, 0);
w.commit();

q = "SELECT * FROM users WHERE login = $1 AND password = $2";

res = w.exec_params(q, login, password);
w.commit();

if(res.empty()){
    pthread_mutex_unlock(&db_mutex);
    return R_FAIL;
}
pthread_mutex_unlock(&db_mutex);
return R_SUCCESS;
}

int StartupDbContext::logout(string login){
    pthread_mutex_lock(&db_mutex);
    if(!isConnected()){
        cout << "NO CON" << endl;
        pthread_mutex_unlock(&db_mutex);
        return -1;
    }

    work w(*conn);
    string q = "UPDATE users SET online = $1, address = null, port = null WHERE login = $2";
    w.exec_params(q, 0, login);
    w.commit();
    pthread_mutex_unlock(&db_mutex);
    return 0;
}

string StartupDbContext::get_lobbies(){
    pthread_mutex_lock(&db_mutex);
    if(!isConnected()){
        cout << "ОШИБКА: НЕТ СОЕДИНЕНИЯ С БАЗОЙ!" << endl;
        pthread_mutex_unlock(&db_mutex);
        return "";
    }

    string q = "SELECT g.id, g.name, g.size, g.busy, g.began, u.login as creator "
               "FROM games g JOIN users u ON g.creator = u.id "
               "ORDER BY g.id";

    work w(*conn);
    result res = w.exec(q);
    w.commit();
    pthread_mutex_unlock(&db_mutex);
    string answer;

```

```

        answer +=
"=====
        answer += "          ID              Название              Создатель              К-во игроков              Иг
началась\n";
        answer +=
"=====

        for(int i = 0; i < res.size(); i++){
            char buff[512];
            bool stat = res[i]["began"].as<bool>();
            snprintf(buff, sizeof(buff), "          %d              %s              %s
%s\n",
                res[i]["id"].as<int>(),
                res[i]["name"].as<string>().c_str(),
                res[i]["creator"].as<string>().c_str(),
                res[i]["busy"].as<int>(),
                res[i]["size"].as<int>(),
                stat ? "*" : " ");
            answer += buff;
            answer += "-----\n";
        }
        return answer;
}

string StartupDbContext::get_players_on(){
    pthread_mutex_lock(&db_mutex);
    if(!isConnected()){
        cout << "ОШИБКА: НЕТ СОЕДИНЕНИЯ С БАЗОЙ!" << endl;
        pthread_mutex_unlock(&db_mutex);
        return "";
    }
    string q = "SELECT * FROM users WHERE online = $1 ORDER BY id";
    work w(*conn);
    result res = w.exec_params(q, 1);
    w.commit();
    pthread_mutex_unlock(&db_mutex);
    string answer;
    answer += "=====\\n";
    answer += "          ID              Логин              \\n";
    answer += "=====\\n";
    for(int i = 0; i < res.size(); i++){
        char buff[256];
        sprintf(buff, "          %d              %s\\n", res[i]["id"].as<int>(),
res[i]["login"].as<string>().c_str());
        answer += buff;
        answer += "-----\\n";
    }
    return answer;
}

string StartupDbContext::get_players_all(){
    pthread_mutex_lock(&db_mutex);
    if(!isConnected()){
        cout << "ОШИБКА: НЕТ СОЕДИНЕНИЯ С БАЗОЙ!" << endl;
        pthread_mutex_unlock(&db_mutex);
        return "";
    }
    string q = "SELECT * FROM users ORDER BY id";
    work w(*conn);
    result res = w.exec(q);
    w.commit();
    pthread_mutex_unlock(&db_mutex);
    string answer;

```

```

answer += "=====\n";
answer += "          ID          Логин          Статус\n";
answer += "=====\n";
for(int i = 0; i < res.size(); i++){
    char buff[256];
    char buff2[20];
    sprintf(buff, "          %d          %s          ", res[i]["id"].as<int>(),
res[i]["login"].as<string>().c_str());
    if(res[i]["online"].as<int>() == 1){
        sprintf(buff2, "%s\n", "Онлайн");
    }else{
        sprintf(buff2, "%s\n", "Не в сети");
    }
    strcat(buff, buff2);
    answer += buff;
    answer += "-----\n";
}
return answer;
}

string StartupDbContext::get_rating(){
    pthread_mutex_lock(&db_mutex);
    if(!isConnected()){
        cout << "ОШИБКА: НЕТ СОЕДИНЕНИЯ С БАЗОЙ!" << endl;
        pthread_mutex_unlock(&db_mutex);
        return "";
    }
    string q = "SELECT * FROM users ORDER BY score DESC, id ASC";
    work w(*conn);
    result res = w.exec(q);
    w.commit();
    pthread_mutex_unlock(&db_mutex);
    string answer;
    answer += "=====\n";
    answer += "      Место      ID          Логин          Рейтинг\n";
    answer += "=====\n";

    int place = 1;
    int score = res[0]["score"].as<int>();

    for(int i = 0; i < res.size(); i++){
        int cscore = res[i]["score"].as<int>();
        if(cscore != score){
            place++;
            score = cscore;
        }
        char buff[256];
        sprintf(buff, "      %d          %d          %s          %d\n", place,
res[i]["id"].as<int>(),
res[i]["login"].as<string>().c_str(), cscore);

        answer += buff;
        answer += "-----\n";
    }
    return answer;
}

int StartupDbContext::add_lobby(string creator, string name, int num){
    pthread_mutex_lock(&db_mutex);
    if(!isConnected()){
        cout << "ОШИБКА: НЕТ СОЕДИНЕНИЯ С БАЗОЙ!" << endl;
        pthread_mutex_unlock(&db_mutex);
        return -1;
    }
}

```

```

}

work w(*conn);

string q1 = "SELECT id FROM users WHERE login = $1";
string q2 = "INSERT INTO games (name, size, busy, creator, began) VALUES ($1, $2, $3, $4,
RETURNING id";
result r = w.exec_params(q1, creator);
w.commit();
if(r.empty()){
    pthread_mutex_unlock(&db_mutex);
    return -1;
}
int cid = r[0]["id"].as<int>();

result res = w.exec_params(q2,name, num, 0, cid, false);
int game_id = res[0]["id"].as<int>();
w.commit();
pthread_mutex_unlock(&db_mutex);
return game_id;
}

int StartupDbContext::join_lobby(int id){
    pthread_mutex_lock(&db_mutex);
    if(!isConnected()){
        cout << "ОШИБКА: НЕТ СОЕДИНЕНИЯ С БАЗОЙ!" << endl;
        pthread_mutex_unlock(&db_mutex);
        return -1;
    }
    string q = "SELECT * FROM games WHERE id = $1";
    work w(*conn);
    result res = w.exec_params(q, id);
    w.commit();
    pthread_mutex_unlock(&db_mutex);
    if(res.empty()){
        return -1;
    }
    return res[0]["port"].as<int>();
}

void StartupDbContext::set_lobby_num(int id, int nv){
    pthread_mutex_lock(&db_mutex);
    if(!isConnected()){
        cout << "ОШИБКА: НЕТ СОЕДИНЕНИЯ С БАЗОЙ!" << endl;
        pthread_mutex_unlock(&db_mutex);
        return;
    }
    string q = "UPDATE games SET busy = $1 WHERE id = $2";
    work w(*conn);
    w.exec_params(q, nv, id);
    w.commit();
    pthread_mutex_unlock(&db_mutex);
}

void StartupDbContext::rm_lobby(int id){
    pthread_mutex_lock(&db_mutex);
    if(!isConnected()){
        cout << "ОШИБКА: НЕТ СОЕДИНЕНИЯ С БАЗОЙ!" << endl;
        pthread_mutex_unlock(&db_mutex);
        return;
    }
    string q = "DELETE FROM games WHERE id = $1";
    work w(*conn);

```

```

w.exec_params(q, id);
w.commit();
pthread_mutex_unlock(&db_mutex);
}

void StartupDbContext::set_lobby_port(int id, int port){
pthread_mutex_lock(&db_mutex);
if(!isConnected()){
    cout << "ОШИБКА: НЕТ СОЕДИНЕНИЯ С БАЗОЙ!" << endl;
    pthread_mutex_unlock(&db_mutex);
    return;
}
string q = "UPDATE games SET port = $1 WHERE id = $2";
work w(*conn);
w.exec_params(q, port, id);
w.commit();
pthread_mutex_unlock(&db_mutex);
}

void StartupDbContext::set_lobby_status(int id, bool ready){
pthread_mutex_lock(&db_mutex);
if(!isConnected()){
    cout << "ОШИБКА: НЕТ СОЕДИНЕНИЯ С БАЗОЙ!" << endl;
    pthread_mutex_unlock(&db_mutex);
    return;
}
string q = "UPDATE games SET began = $1 WHERE id = $2";
work w(*conn);
w.exec_params(q, ready, id);
w.commit();
pthread_mutex_unlock(&db_mutex);
}

void StartupDbContext::add_player_score(string login, int score){
pthread_mutex_lock(&db_mutex);
if(!isConnected()){
    cout << "ОШИБКА: НЕТ СОЕДИНЕНИЯ С БАЗОЙ!" << endl;
    pthread_mutex_unlock(&db_mutex);
    return;
}
string q = "UPDATE users SET score = $1 WHERE login = $2";
string q1 = "SELECT * FROM users WHERE login = $1";
work w(*conn);
result r = w.exec_params(q1, login);
w.commit();
result res = w.exec_params(q, r[0]["score"].as<int>() + score, login);
w.commit();
pthread_mutex_unlock(&db_mutex);
}

void StartupDbContext::clear_online(){
pthread_mutex_lock(&db_mutex);
if(!isConnected()){
    cout << "ОШИБКА: НЕТ СОЕДИНЕНИЯ С БАЗОЙ!" << endl;
    pthread_mutex_unlock(&db_mutex);
    return;
}
string q = "UPDATE users SET online = $1, address = null, port = null";
work w(*conn);
w.exec_params(q, 0);
w.commit();
pthread_mutex_unlock(&db_mutex);
}

```

```

}

void StartupDbContext::clear_lobbies() {
    pthread_mutex_lock(&db_mutex);
    if(!isConnected()){
        cout << "ОШИБКА: НЕТ СОЕДИНЕНИЯ С БАЗОЙ!" << endl;
        pthread_mutex_unlock(&db_mutex);
        return;
    }
    string q = "TRUNCATE TABLE games";
    work w(*conn);
    w.exec(q);
    w.commit();
    pthread_mutex_unlock(&db_mutex);
}

bool StartupDbContext::finduser(string nick, string& ip, int& port, bool& online){
    pthread_mutex_lock(&db_mutex);
    if(!isConnected()){
        cout << "ОШИБКА: НЕТ СОЕДИНЕНИЯ С БАЗОЙ!" << endl;
        pthread_mutex_unlock(&db_mutex);
        return false;
    }
    string q = "SELECT * FROM users WHERE login = $1";
    work w(*conn);
    result res = w.exec_params(q, nick);
    w.commit();
    pthread_mutex_unlock(&db_mutex);
    if(res.empty()){
        return false;
    }
    if(res[0]["online"].as<int>() == 0){
        online = false;
    }else{
        online = true;
        ip = res[0]["address"].as<string>();
        port = res[0]["port"].as<int>();
    }
    return true;
}

bool StartupDbContext::setupport(string login, int port){
    pthread_mutex_lock(&db_mutex);
    if(!isConnected()){
        cout << "ОШИБКА: НЕТ СОЕДИНЕНИЯ С БАЗОЙ!" << endl;
        pthread_mutex_unlock(&db_mutex);
        return false;
    }
    string q = "UPDATE users SET port = $1 WHERE login = $2";
    work w(*conn);
    w.exec_params(q, port, login);
    w.commit();
    string q2 = "SELECT * FROM users WHERE login = $1 AND port = $2";
    result res = w.exec_params(q2, login, port);
    w.commit();
    pthread_mutex_unlock(&db_mutex);
    if(res.empty()){
        return false;
    }
    return true;
}

```

## **Список источников**

1. **Павский, К. В.** Протоколы TCP/IP и разработка сетевых приложений : учебное пособие / К. В. Павский ; Сибирский государственный университет телекоммуникаций и информатики. – Новосибирск : СибГУТИ, 2013. – 131 с. : ил. – Текст : электронный. – URL: [http://ellib.sibsutis.ru/ellib/2013\443\\_Pavskij\\_K.V.\\_Protokoly\\_TCP\\_.rar](http://ellib.sibsutis.ru/ellib/2013\443_Pavskij_K.V._Protokoly_TCP_.rar) (дата обращения: 16.04.2026). – Режим доступа: по паролю.
2. **Доусон, М.** Изучаем C++ через программирование игр / М. Доусон ; перевод с английского Е. Зазноба, О. Сивченко. — Санкт-Петербург : Питер, 2020. — 352 с. — ISBN 978-5-4461-1791-8. — Текст : непосредственный.